

Mesh-agnostic prediction of unsteady flow dynamics using graph U-Nets

Sunwoong Yang ^{a,b}, Ricardo Vinuesa ^{c,d}, Namwoo Kang ^{a,e,*}

^a Cho Chun Shik Graduate School of Mobility, Korea Advanced Institute of Science and Technology, Daejeon, 34051, Republic of Korea

^b Department of Mechanical Engineering, Hanyang University, Ansan, 15588, Republic of Korea

^c Department of Aerospace Engineering, University of Michigan, Ann Arbor, MI, 48109, United States

^d FLOW, Engineering Mechanics, KTH Royal Institute of Technology, Stockholm, SE-100 44, Sweden

^e Narnia Labs, Daejeon, 34051, Republic of Korea

ARTICLE INFO

Keywords:

Scientific machine learning
Mesh-agnostic flow prediction
Spatio-temporal unsteady flow prediction
Graph neural networks
Transductive learning
Inductive learning

ABSTRACT

This study presents a comprehensive investigation of U-Net-based graph neural networks (Graph U-Nets) for mesh-agnostic spatio-temporal forecasting of unsteady flow fields. We systematically adapt and enhance Graph U-Nets, originally developed for classification tasks, for high-dimensional regression problems in fluid dynamics through extensive architectural modifications and hyperparameter optimization. Key enhancements include the implementation of Gaussian mixture model convolutional operators, which provide superior flexibility in modeling node dynamics and reduce prediction error by 95% compared to conventional graph convolutional operators. Additionally, we introduce noise injection strategies that significantly improve long-term prediction robustness, achieving an 86% reduction in temporal prediction error. Through comprehensive ablation studies, we investigate the effects of pooling strategies, normalization techniques, and architectural choices on model performance. We demonstrate the framework's effectiveness in both transductive learning settings—successfully predicting flow fields in unseen spatial regions—and inductive learning scenarios across diverse mesh configurations with varying vortex shedding dynamics. Notably, we discover that optimal inductive performance requires eliminating pooling operations and employing layer normalization, contrary to single-mesh scenarios. The enhanced Graph U-Net successfully generalizes to unseen mesh scenarios, achieving improved prediction accuracy for challenging slow-vortex-shedding cases when trained on diverse flow regimes. This work establishes Graph U-Nets as a viable and flexible alternative to convolutional neural networks for computational fluid dynamics applications, demonstrating their potential for real-time flow prediction in digital twin frameworks across diverse industrial scenarios.

1. Introduction

Instantaneous transient flow prediction—forecasting flow properties such as velocity and pressure that evolve over time—is essential for the realization of digital twins, which serve as the cornerstone for real-time forecasting and decision making in a variety of engineering fields. More specifically, the integration of accurate unsteady flow prediction models is critical to the fidelity and effectiveness of digital twins, enabling them to mirror their physical counterparts in real-time. However, conventional high-fidelity computational fluid dynamics (CFD) analysis, which is one of the most useful approaches for understanding fluid-flow behavior, has the limitation of requiring significant computational resources for practical applications (Hu et al., 2022; Qu et al., 2022).

In this context, artificial intelligence (AI) approaches have emerged as promising alternatives, playing a critical role in predicting real-time

flow fields within digital-twin scenarios (Barnett et al., 2023; Barwey et al., 2023; Eivazi et al., 2022b; Jeon et al., 2024; Vinuesa, 2024; Yang et al., 2024). Leveraging the power of AI can significantly reduce the computational burden while approximating the accuracy of CFD simulations and automating the entire analysis process. In addition, AI-driven approaches offer the potential for real-time prediction, which is essential for digital twins that can adapt to dynamic flow conditions. These advances pave the way for more sophisticated and responsive digital twins, ensuring that their digital counterparts can operate in real-time with satisfactory accuracy and efficiency, revolutionizing predictive maintenance and operational effectiveness across CFD-related industries.

Particularly, convolutional neural networks (CNNs) have led to significant advances in the prediction of both steady and unsteady flow fields (Eivazi et al., 2022a; Guastoni et al., 2021; Hasegawa et al., 2020; Hu & Zhang, 2022; Kang et al., 2022; Maulik et al., 2021) (detailed

* Corresponding author.

E-mail addresses: sunwoongy@hanyang.ac.kr (S. Yang), rvinuesa@mech.kth.se (R. Vinuesa), nwkang@kaist.ac.kr (N. Kang).

<https://doi.org/10.1016/j.eswa.2026.131843>

Received 23 September 2024; Received in revised form 21 February 2026; Accepted 23 February 2026

Available online 4 March 2026

0957-4174/© 2026 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

in Section 2). While CNNs have introduced more efficient and physically meaningful flow-prediction algorithms due to their spatial locality property, they are inherently designed for structured grids and therefore face significant limitations when confronted with unstructured meshes. Although it is possible to apply CNNs to unstructured data by first interpolating the field data onto a uniform structured grid, this approach often leads to a decrease in prediction accuracy due to interpolation errors and requires a pre-processing procedure to select an appropriate interpolation method. This limitation is exacerbated as unstructured meshes become more prevalent in real-world industrial sites due to their flexibility in handling complex geometries and dynamic mesh scenarios. Consequently, the reliance on CNNs for mesh-based predictions highlights a significant limitation of their potential within digital twins for real-world industrial scenarios. This gap highlights the need for advanced deep-learning models that can handle the various unstructured grid systems for generality and versatility in practical applications.

In that regard, graph neural networks (GNNs) can be a powerful solution, effectively bridging the gap left by CNNs through their inherent mesh scalability (Barwey et al., 2023; Chen et al., 2024; Gao et al., 2023; Lino et al., 2022; Ogoke et al., 2021; Pichi et al., 2024; Wang et al., 2024; Xiang et al., 2024; Yang et al., 2025b) (detailed in Section 2). They can handle different structures of mesh datasets by learning directly from the graph consisting of nodes and edges without the spatial topology restrictions (structured grid) inherent in CNNs (Roznowicz et al., 2024). This adaptability makes them inherently mesh-agnostic, meaning a single trained model can make predictions on various unseen mesh configurations without requiring retraining. Therefore, GNNs are effectively positioned to address the challenges associated with both structured and unstructured meshes commonly encountered in CFD simulations. The mesh flexibility of GNNs enables them to seamlessly adapt to various mesh configurations during training and inference, making them highly compatible with diverse CFD datasets. By directly operating on graph-structured data, GNNs can effectively capture the intricate spatial relationships and dependencies present in the mesh, allowing them to model the complex dynamics and evolution of unsteady flow fields with high fidelity.

Among the various approaches based on GNNs, Graph U-Nets, which originated from the U-Net architecture, have been introduced for their performance in node/graph classification tasks (Gao & Ji, 2019). U-Nets have been widely adopted for their efficiency in feature extraction and localization across different resolution scales, enabling precise segmentation and reconstruction tasks (Anand et al., 2023; Cremades et al., 2024; Deng et al., 2022; Feng et al., 2021; Jurado et al., 2022; Ronneberger et al., 2015; Song et al., 2024; Tang et al., 2020) (detailed in Section 2). This ability of the U-Nets to capture multi-scale features is particularly well-suited for flow-prediction tasks where flow structures with a wide range of length scales exist. By integrating the U-Net framework with GNNs, Graph U-Nets inherit these strengths, enabling successful performance even in untrained mesh scenarios (Gao & Ji, 2019). However, their comprehensive validation has not been conducted in regression tasks with high-dimensional spatio-temporal output space, which are the most common in practical flow problems. Therefore, this work attempts to explore the potential of this combination, U-Nets and GNNs, which would ensure the multi-scale nature of U-Nets even in the unstructured-data situation, allowing them to capture both local and global flow patterns and make them highly effective in predicting transient flow fields with complex spatio-temporal dependencies.

This study focuses on leveraging the Graph U-Net architecture to predict vortex-shedding flow over a circular cylinder, which is one of the most famous benchmark flow problems for spatio-temporal prediction (Patel, 1978). We propose several novel approaches to augment the synergy between the strengths of GNNs in capturing mesh-based dependencies and the spatio-temporal dynamics of transient flows. The proposed methods introduce several key enhancements to the traditional Graph U-Net architecture, including the Gaussian mixture model (GMM) convolutional operator (Monti et al., 2017) for increased flexibility in mod-

eling node dynamics, and noise injection approaches (Pfaff et al., 2020) to mitigate error accumulation over long time-stepping. Throughout this study, extensive parametric studies that serve as comprehensive ablation analyses are conducted to investigate the impact of various architectural choices and hyperparameters on the performance of the Graph U-Net model in order to optimize its performance specifically for fluid dynamics applications.

The main contributions of this study can be summarized as follows:

1. A comprehensive exploration of Graph U-Nets in unsteady flow-field prediction, bridging the gap between structured grid-based CNN approaches and the need for flexible, mesh-agnostic real-time prediction models in CFD domain.
2. Systematic investigation and optimization of Graph U-Net architecture for fluid dynamics applications, including:
 - a) Implementation of the Gaussian mixture model (GMM) convolutional operator, demonstrating increased flexibility and accuracy in modeling node dynamics.
 - b) Analysis of pooling strategies, revealing their impact on prediction accuracy and generalization capabilities.
 - c) Implementation of noise injection techniques to significantly enhance long-term prediction stability.
 - d) Comparative assessment of normalization methods, uncovering their crucial role in model performance across different mesh scenarios.
3. In-depth investigation of both transductive and inductive learning performance, demonstrating the ability of Graph U-Nets to generalize to untrained node regions and entirely new mesh configurations.
4. Novel insights into inductive learning for Graph U-Nets, including:
 - a) Successful extension of the training scenario to encompass meshes with varying vortex shedding periods.
 - b) The counter-intuitive finding that omitting pooling operations leads to superior performance in inductive settings.
 - c) Discovery of the significant impact of normalization techniques on inductive learning, with the optimal method depending on the physical characteristics of the graph scenarios being evaluated.
5. Demonstration of substantial improvements in prediction accuracy and long-term stability through carefully tuned Graph U-Net models, establishing their viability for fluid dynamics applications.
6. Development of a comprehensive framework for adapting and optimizing Graph U-Nets, originally designed primarily for classification tasks, to perform effectively in regression tasks for spatio-temporal flow prediction. This work demonstrates the versatility of Graph U-Nets beyond their conventional classification applications, unlocking new potential for their use in complex flow dynamics predictions.

The remainder of this paper is organized as follows. Section 3 details the methodologies employed, including the Graph U-Net architecture, its key components like the convolutional operators, and the normalization techniques. Section 4 describes the experimental setup, the dataset used for training, and the evaluation process. In Section 5, we present an in-depth analysis of the enhancements to the Graph U-Net, including the effects of different convolutional operators, pooling ratios, and noise injection schemes. Section 6 evaluates the transductive and inductive learning performance of the optimized models across various mesh scenarios. Finally, Section 7 concludes the paper with a summary of our key findings and a discussion of future work.

2. Related work

CNNs. As noted in the introduction section, CNNs have led to significant advances in the prediction of both steady and unsteady flow fields. For instance, Kang et al. (2022) developed a physics-aware reduced-order model for steady transonic flow over an airfoil, using a β -variational autoencoder (β -VAE) with a CNN architecture to handle structured mesh data and extract physically interpretable latent variables. Maulik et al. (2021) combined a convolutional autoencoder for

nonlinear dimensionality reduction with a long short-term memory network for time-series evolution to model advection-dominated systems, such as the viscous Burgers and inviscid shallow water equations. Similarly, Hasegawa et al. (2020) constructed a reduced-order model for unsteady flows around various bluff body shapes by first using a multi-scale CNN-based autoencoder to map high-dimensional flow fields to a low-dimensional latent space, and then employing an LSTM to predict the temporal dynamics of the resulting vectors. Guastoni et al. (2021) utilized fully convolutional networks (FCNs) to predict instantaneous velocity fluctuation fields in turbulent open-channel flow from wall-shear-stress and pressure data, comparing a direct prediction model, FCN, with one that predicts POD coefficients (FCN-POD). Eivazi et al. (2022a) proposed a deep probabilistic model using a CNN-based β -VAE to extract a minimal and near-orthogonal set of non-linear modes from unsteady turbulent flow data in a simplified urban environment. Lastly, to address the limitations of standard CNNs on non-uniform grids, Hu and Zhang (2022) introduced a novel “Mesh-Conv” operator that allows CNNs to directly model flow fields on non-uniform structured meshes without pixelation by incorporating local weightings based on mesh geometry.

GNNs. In that regard, GNNs have emerged as a powerful solution, effectively bridging the gap left by CNNs through their inherent mesh scalability. For example, Ogoke et al. (2021) applied a graph convolutional network (GCN) with inductive layers to predict the scalar drag force on various airfoils in laminar flow from velocity fields on unstructured meshes. Addition to this, autoencoder architectures have been developed for model reduction, such as the graph convolutional autoencoder proposed by Pichi et al. (2024) to handle parametrized PDEs on complex geometries. To better capture phenomena spanning a range of length scales, multi-scale architectures have been introduced; Lino et al. (2022) proposed a multi-scale rotation-equivariant GNN that processes information through graph coarsening to forecast unsteady Eulerian fluid dynamics, while Barwey et al. (2023) developed a multiscale GNN autoencoder that uses adaptive pooling to create interpretable latent graphs for turbulent flow over a backward-facing step. GNNs have also been tailored for specific feature identification tasks, as Wang et al. (2024) used a GNN with a U-Net architecture to directly identify vortices in unstructured CFD meshes within backward-facing step flow. Furthermore, to embed physical constraints, Xiang et al. (2024) introduced a physics-informed GNN framework that integrates a MeshGraphNet with the radial basis function finite difference method to solve spatiotemporal PDEs including heat, wave, and shallow water equations on irregular domains. For specific forecasting applications, GNNs have been developed to handle dynamic spatio-temporal dependencies; for example, Gao et al. (2023) proposed a dynamic adaptive spatio-temporal graph neural network (DASTGN) to predict future multi-node offshore wind speed from historical data. Also, Chen et al. (2024) developed a time-based adaptive GNN (TAGNN) that uses a graph learning module and temporal convolutions to forecast future traffic flow, speed, and occupancy based on past sensor readings.

U-Nets. U-Net architectural concept was pioneered by Ronneberger et al. (2015), who introduced it as a fully convolutional network for the semantic segmentation of biomedical images, a pixel-wise classification task. This foundational work has inspired numerous applications in medical imaging; for example, Anand et al. (2023) presented the fusion U-Net for the segmentation task of identifying brain tumors from multi-modal MRI data, while Tang et al. (2020) proposed an attention-guided dense U-Net (AD-UNet) for the classification task of liver segmentation from CT scans. Similarly, Jurado et al. (2022) successfully employed a standard CNN-based U-Net architecture for the semantic segmentation of retinal layers in optical coherence tomography images. Beyond medical imaging, U-Nets have been adapted for complex physics problems. Feng et al. (2021), for instance, introduced UR-Net, a residual CNN-based U-Net, for the regression task of predicting velocity correction fields to accelerate incompressible turbulence simulations. For feature identification in fluid flows, Deng et al. (2022) proposed a multi-scale-input U-Net (MSI-U-Net) that performs a pixel-wise classification

to identify vortices of various sizes. In the context of spatio-temporal forecasting, Song et al. (2024) developed a two-stage CNN-based U-Net (TS-U-Net) that performs a regression task to predict the long-term evolution of a 2D temperature field. The architecture has also been applied in geoinformatics, where Cremades et al. (2024) utilized a CNN-based U-Net for the semantic segmentation task of identifying urban heat islands from multi-spectral satellite data.

3. Methodologies

3.1. Graph convolutional operators

This section first reviews the graph convolutional network (GCN) operator and then introduces the Gaussian mixture model (GMM) operator as an enhanced alternative.

3.1.1. Graph convolutional network (GCN) operator

GCNs adapt the convolutional operations of CNNs for non-Euclidean, graph-structured data (Kipf & Welling, 2016; Wu et al., 2024). Whereas CNNs operate on regular grids, GCNs aggregate local neighborhood information from nodes and their connections to extract features, capturing spatial hierarchies in irregular data layouts. The conventional GCN operation in node-wise formulation is as follows:

$$\mathbf{x}'_i = \Theta^\top \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{e_{j,i}}{\sqrt{\hat{d}_j \hat{d}_i}} \mathbf{x}_j, \quad (1)$$

where $\mathbf{x}'_i$ represents the updated features for node i , $\mathcal{N}(i) \cup \{i\}$ denotes the set of neighbors for node i including itself (meaning $A + I$ for adjacency matrix A), $e_{j,i}$ is the edge weight from node j to i , $\hat{d}_i$ is the node degree adjusted for edge weights ($\hat{d}_i = 1 + \sum_{j \in \mathcal{N}(i)} e_{j,i}$), and Θ is a learnable weight matrix. This formulation aggregates neighbor features ($\mathbf{x}_j$) normalized by node degrees and weighted by their connections.

The original Graph U-Net paper (Gao & Ji, 2019) proposed an improved GCN layer that uses $A + 2I$ for the adjacency matrix instead of the conventional $A + I$, which assigns greater weight to a node's own features during aggregation. The inclusion of edge weights ($e_{j,i}$) is optional; they can provide information such as the distance between nodes or be omitted (i.e., set to 1) if not relevant. In our comparative analysis (Section 5.1), we test three GCN variants: a conventional GCN (without self-loop improvement or weights), an improved GCN (iGCN) with the self-loop enhancement, and an improved, weighted GCN (iwGCN) that also uses Euclidean distance for edge weights.

3.1.2. Gaussian mixture model (GMM) operator

The GCN approach exhibits limitations in considering edge features, which are crucial for capturing the complex interactions and information propagation in domains like CFD. In such problems, edge features often represent physical distances, carrying valuable information about spatial relationships within the mesh. To address this, the GMM convolutional operator provides a more sophisticated method for incorporating edge attributes (Monti et al., 2017).

While a GCN can naively use Euclidean distance for an edge attribute (by setting $e_{j,i}$ in Eq. (1) to $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$), the GMM operator employs a mixture of Gaussians as a learnable weight function (or kernel). It flexibly learns how to weight edge attributes for a specific task by training a mean vector and diagonal covariance matrix for each kernel. This adaptability allows the GMM to assign variable importance to edges based on the relative positions of neighboring nodes, making it highly effective at capturing the local geometry and complex flow patterns in graph-structured CFD data. The number of kernels is a key hyperparameter whose effects are investigated in Section 5.1. The node-wise formulation of the GMM operator is:

$$\mathbf{x}'_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \frac{1}{K} \sum_{k=1}^K \mathbf{w}_k(\mathbf{e}_{i,j}) \odot \Theta_k \mathbf{x}_j, \quad (2)$$

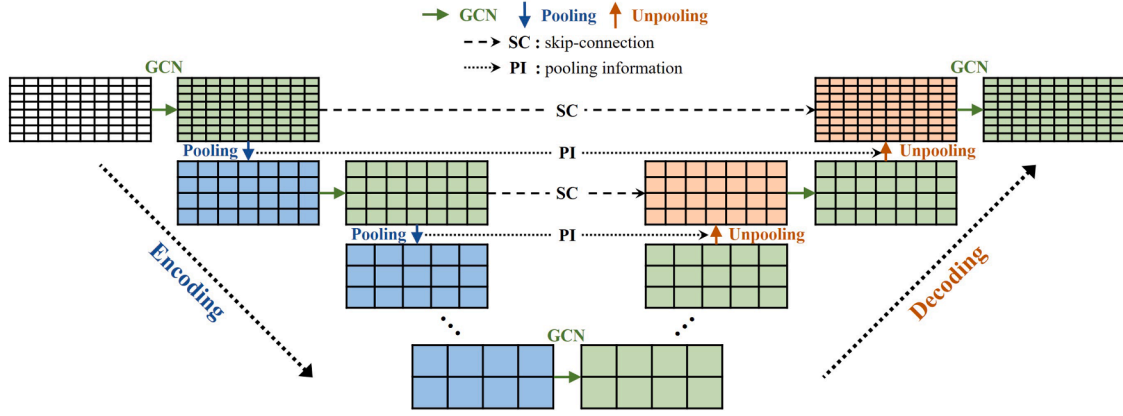


Fig. 1. Graph U-Net architecture, where SC and PI denote skip-connection and pooling information respectively.

where K is the number of kernels, $\mathbf{e}_{i,j}$ is the edge attribute (Euclidean distance in this study) between nodes i and j , and

$$\mathbf{w}_k(\mathbf{e}) = \exp\left(-\frac{1}{2}(\mathbf{e} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{e} - \boldsymbol{\mu}_k)\right) \quad (3)$$

denotes a kernel based on a trainable mean vector $\boldsymbol{\mu}_k$ and a diagonal covariance matrix $\boldsymbol{\Sigma}_k$. The size of these parameters is determined by the pseudo-coordinate dimensionality, which is set to 1 throughout this study (Monti et al., 2017). By learning $\boldsymbol{\mu}_k$ and $\boldsymbol{\Sigma}_k$, the GMM gains additional degrees of freedom compared to a conventional GCN, allowing for a richer representation of graph topologies and enhancing the model's ability to predict intricate data patterns.

3.2. Graph U-Nets

The U-Net architecture features a symmetric encoder-decoder structure with skip-connections (Ronneberger et al., 2015). The encoder progressively downsamples the input to capture context, while the decoder upsamples it to restore spatial details. The skip-connections fuse low-level and high-level feature maps, enabling precise localization and multi-resolution analysis.

Graph U-Nets adapt this powerful concept for non-Euclidean graph data (Gao & Ji, 2019). As illustrated in Fig. 1, the architecture is similar, but it replaces standard CNN operators with GCN operators. Furthermore, conventional grid-based pooling operations (e.g., max pooling) are incompatible with irregular graph structures. The lack of a fixed node ordering makes it challenging to define a consistent unpooling operation, as the target dimensions are unknown (Gao & Ji, 2019). Consequently, Graph U-Nets require specialized pooling and unpooling techniques designed for graph data.

3.2.1. Pooling for graph datasets

To address the challenge of downsampling graphs, Gao and Ji (2019) proposed the graph pooling (gPool) technique, described in Algorithm 1. This method adaptively selects a subset of nodes to create a coarsened graph. Specifically, the algorithm first projects the input feature matrix X^ℓ onto a trainable vector p (line 1 in Algorithm 1). The nodes are then ranked according to the resulting projection scores, and the indices of the top- k nodes are selected (line 2). In this study, k is defined as a ratio of the original number of nodes, such that the ratio of 0.3 means that the top 30% of the nodes of the original graph are preserved. An activation function σ is then applied to the scores of the selected nodes (line 3). To mitigate node isolation, the adjacency matrix of the new graph is updated by considering the two-hop neighborhood of each node (lines 4-5).¹ Finally, the selected node features (line 6) are gated by their cor-

responding scores to produce the feature matrix for the coarsened graph (line 7). This entire process allows the gPool layer to learn a hierarchical representation by training the projection vector p via back-propagation (LeCun et al., 2002).

Algorithm 1 Graph pooling (gPool) layer.

Input: Graph G^ℓ with adjacency matrix $A^\ell \in \mathbb{R}^{N \times N}$ and feature matrix $X^\ell \in \mathbb{R}^{N \times F}$, pooled nodes k , trainable projection vector $p \in \mathbb{R}^F$, and activation function σ

Output: Coarsened graph $G^{\ell+1}$ with adjacency matrix $A^{\ell+1} \in \mathbb{R}^{k \times k}$ and feature matrix $X^{\ell+1} \in \mathbb{R}^{k \times F}$

- 1: $y \leftarrow X^\ell \cdot p / |p|$ ▷ Project features onto p
- 2: $idx \leftarrow \text{rank}(y, k)$ ▷ Get indices of top k nodes
- 3: $\tilde{y} \leftarrow \sigma(y(idx))$ ▷ Apply activation to selected nodes
- 4: $A^2 \leftarrow A^\ell A^\ell$ ▷ Compute the second-power of adjacency matrix
- 5: $A^{\ell+1} \leftarrow A^2(idx, idx)$ ▷ Update adjacency matrix of coarsened graph
- 6: $\tilde{X}^\ell \leftarrow X^\ell(idx, :)$ ▷ Select features of top k nodes
- 7: $X^{\ell+1} \leftarrow \tilde{X}^\ell \odot (\tilde{y} \cdot \mathbf{1}_F^T)$ ▷ Apply gating to selected features
- 8: **return** $A^{\ell+1}, X^{\ell+1}$

3.2.2. Unpooling for graph datasets

The Graph Unpooling (gUnpool) layer is a critical component of the decoder, performing the inverse operation of gPool to restore the original graph structure (Gao & Ji, 2019). Since recovering the original dimensions of irregular graph data is impossible without additional information, the gUnpool layer relies on information passed from the corresponding gPool layer in the encoder: in Fig. 1, this information is presented as PI (pooling information).

The process is detailed in Algorithm 2. To upsample a coarsened graph $G^{\ell+1}$ back to its original state G^ℓ , the layer uses the node count (N_{saved}), node indices (idx_{saved}), and adjacency matrix (A_{saved}^ℓ) that were saved during the corresponding gPool operation. First, it initializes a zero-feature matrix with the original graph's dimensions (line 1 in Algorithm 2). It then uses the saved indices to place the features from the coarsened graph back into their correct positions (line 2). Finally, the original adjacency matrix is restored (line 3). This seamless integration of gPool and gUnpool provides the necessary mapping to accurately reconstruct the graph structure, enabling the Graph U-Net to learn hierarchical representations and capture both local and global patterns across different resolutions.

¹ In this study, the two-hop neighborhood is not considered. The original purpose of using the second-power adjacency matrix is to compensate for the signif-

icantly coarsened graph during pooling. However, in the domain of CFD, nodes in the graph (mesh) are usually highly connected to each other, so this approach rather tends to increase the number of edges after pooling — this result is ironic when we recall that the purpose of pooling is to coarsen the graph to capture features at a larger scale.

Algorithm 2 Graph unpooling (gUnpool) layer.

Input: Graph to be upsampled $G^{\ell+1}$ with feature matrix $X^{\ell+1} \in \mathbb{R}^{k \times F}$ & information of the original graph G^ℓ : node size N_{saved} , node indices $\text{id}x_{\text{saved}}$, and adjacency matrix A_{saved}^ℓ saved at the corresponding gPool layer

Output: Upsampled graph G^ℓ with feature matrix $X^\ell \in \mathbb{R}^{N \times F}$ and adjacency matrix A^ℓ

- 1: $X^\ell \leftarrow \mathbf{0}_{N_{\text{saved}} \times F}$ $\triangleright$ Initialize output feature matrix with zeros
- 2: $X^\ell(\text{id}x_{\text{saved}}, :) \leftarrow X^{\ell+1}$ $\triangleright$ Fill the new features with the previous features
- 3: $A^\ell \leftarrow A_{\text{saved}}^\ell$ $\triangleright$ Update the adjacency matrix
- 4: **return** A^ℓ, X^ℓ

3.3. Normalization methods

Generalizing to new graphs not seen during training, or inductive learning, is challenging for GNNs when training on datasets with heterogeneous physical properties and mesh structures. The diversity of node features across different graphs can hinder model generalization, necessitating robust feature scaling methods. This section demonstrates the role of normalization methods, namely layer normalization (LN) and graph normalization (GN), in enhancing the predictive performance of Graph U-Nets in inductive settings.

3.3.1. Layer normalization (LN)

LN, detailed in [Algorithm 3](#), effectively addresses internal covariate shift by normalizing the features for each individual node ([Ba et al., 2016](#)). This technique operates independently on each node, standardizing its feature vector (the time-series of x -velocity in this study) without considering inter-node relationships. This node-centric approach is crucial for preserving unique local dynamics and is independent of the mini-batch size ([Ba et al., 2016](#)). Except where otherwise noted in [Section 6.2.3](#), all models in this study adopt LN.

Algorithm 3 Layer normalization (LN).

Input: Node features $X \in \mathbb{R}^{N \times F}$ for N nodes each with F features

Output: Normalized features $X' \in \mathbb{R}^{N \times F}$

- 1: Compute the mean μ_i and variance σ_i^2 across all features:

$$\mu_i = \frac{1}{F} \sum_{j=1}^F X_{ij}, \quad \sigma_i^2 = \frac{1}{F} \sum_{j=1}^F (X_{ij} - \mu_i)^2$$

$\triangleright i$ is the index for each node

- 2: Normalize the features:

$$\hat{X}_{ij} = \frac{X_{ij} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \quad \text{for all } i, j$$

$\triangleright j$ is the index for each feature and ϵ is the hyperparameter for numerical stability

- 3: Scale and shift the normalized features:

$$X'_{ij} = \gamma_j \hat{X}_{ij} + \beta_j \quad \text{for all } i, j$$

$\triangleright \gamma_j$ and β_j are learnable parameters for each feature dimension

- 4: **return** X' $\triangleright$ Return the layer-normalized feature matrix

3.3.2. Graph normalization (GN)

GN, detailed in [Algorithm 4](#), is designed specifically for graph data ([Cai et al., 2021](#)). In contrast to LN, GN normalizes each feature across all nodes in a graph, rather than normalizing each node's features independently (see line 1 in [Algorithms 3](#) and [4](#)). GN computes statistics across the node dimension and introduces a learnable parameter, α_j , that weights the mean during the shift operation (lines 1-2 in [Algorithm 4](#)). This learnable shift was shown by [Cai et al. \(2021\)](#) to prevent

the expressive degradation found in other methods like InstanceNorm ([Ulyanov et al., 2016](#)), leading to strong performance on graph classification benchmarks.

Algorithm 4 Graph normalization (GN).

Input: Node features $X \in \mathbb{R}^{N \times F}$ for N nodes each with F features

Output: Normalized features $X' \in \mathbb{R}^{N \times F}$

- 1: Compute the mean μ_j and variance σ_j^2 across all nodes:

$$\mu_j = \frac{1}{N} \sum_{i=1}^N X_{ij}, \quad \sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (X_{ij} - \alpha_j \cdot \mu_j)^2$$

$\triangleright j$ is the index for each feature and α_j is the learnable parameter for each feature dimension

- 2: Normalize the features:

$$\hat{X}_{ij} = \frac{X_{ij} - \alpha_j \cdot \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad \text{for all } i, j$$

$\triangleright i$ is the index for each node and ϵ is the hyperparameter for numerical stability

- 3: Scale and shift the normalized features:

$$X'_{ij} = \gamma_j \hat{X}_{ij} + \beta_j \quad \text{for all } i, j$$

$\triangleright \gamma_j$ and β_j are learnable parameters for each feature dimension

- 4: **return** X' $\triangleright$ Return the graph-normalized feature matrix

4. Details of experiments

This section details the training process for the Graph U-Net models, which are applied to predict vortex shedding behind a two-dimensional (2D) circular cylinder. We use a public benchmark dataset from Google DeepMind ([Pfaff et al., 2020](#)) for evaluating our graph-based methods in fluid dynamics ([Deng et al., 2024](#); [NVIDIA Corporation, 2025](#)). The dataset contains multiple scenarios with varying cylinder geometries and flow conditions, generated using the COMSOL incompressible Navier-Stokes solver ([Multiphysics, 2015](#)). Each simulation ran for 6 s with a 0.01-s time step, yielding 600 snapshots² per each mesh scenario.

Our initial experiments focus on a single mesh scenario, using 150 consecutive snapshots for training. This baseline case, shown in [Fig. 2](#), features a mesh with 1,946 nodes and 11,208 edges. Its flow is characterized by a maximum inlet velocity³ of 1.78 m/s and a cylinder diameter of 0.074 m, resulting in a vortex shedding period of 29 snapshots (see [Table 5](#) for full details). The graph U-Net model is trained to predict the next snapshot using a sequence of the 20 preceding snapshots as input (i.e., 20 input channels). The loss function is the mean-squared error (MSE) between the prediction and the ground truth, aggregated over the training set for backpropagation. During inference, future snapshots are generated via an auto-regressive manner ([Fig. 3](#)), where the model's prediction at each step becomes an input for the subsequent step.

Due to GPU memory constraints, training focused exclusively on the x -velocity component. However, the model is capable of handling multi-channel inputs to predict the complete flow field (e.g., all velocity components and pressure) given additional computational resources. The Graph U-Net architecture incorporates 4 graph convolutional network (GCN) layers in an encoder-decoder structure ([Gao & Ji, 2019](#)). In the encoder, each GCN layer is followed by a gPool layer with a pooling ratio of 0.5, reducing the number of nodes by 50% at each step. The original graph, with 1,946 nodes and 20 input channels, is thus transformed through the encoder. After the initial GCN layer (coupled with

² A snapshot means the spatial distribution of flow variables at a particular instant.

³ Since the x -velocity profile of the inlet is assumed to be parabolic, the midpoint of the profile, which is the maximum value, is presented.

Table 1
Summary of hyperparameters and architectural choices.

Hyperparameter	Investigated Range	Optimal Value Used
General Settings		
Optimizer	–	Adam
Learning Rate	–	10^{-3} (with scheduler)
Epochs	–	7500 (with early stopping)
Activation Function	–	ELU
Number of Conv. Layers	–	4
Single-Mesh Experiments (Section 5)		
Conv. Operator	GCN, iGCN, iwGCN, GMM	GMM
Number of GMM Kernels	1, 2, 3	1
Pooling Ratio	[0.1, 0.2, ..., 0.9], None	0.6
Noise Injection Size (σ)	[0.001, ..., 0.64]	0.16
Number of Input Snapshots (N)	–	20
Inductive Learning Experiments (Section 6)		
Number of GMM Kernels	1, 3	3
Pooling Ratio	[0.2, 0.4, 0.6, 0.8], None	None
Number of Input Snapshots	1, 5, 10, 20, 40	40

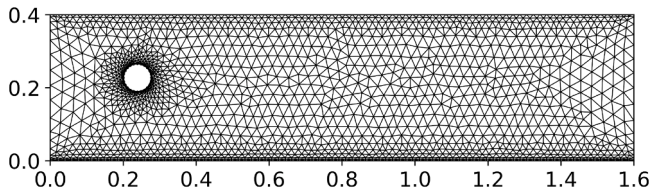


Fig. 2. Mesh used for training, containing 1946 nodes, 11,208 edges, and 3658 volume cells. The flow is from left to right and the details of the flow condition can be found in Table 5 (named as baseline graph).

LN and an ELU activation function (Clevert et al., 2015)), the number of channels is set to decrease as 20, 15, 10, 5, and 1 across the 4 GCN layers, reducing the 20-channel input to a single channel before the decoder. By progressively reducing dimensionality, this hierarchical pooling strategy enables the capture of multi-scale dependencies crucial for complex fluid flows. At the end of the encoding phase, a final GCN layer acts as a bridge to the decoder.

The decoder reconstructs the flow field through a reverse sequence. An unpooling operation first restores the number of nodes, using information transferred from the corresponding gPool layer. Decoder blocks

then benefit from skip-connections that pass features from the corresponding encoder level (Fig. 1). A GCN layer is also applied to the unpooled graph, integrating features from the previous decoder level with those passed through the skip-connection. This unpooling-GCN block is repeated three more times with a single channel, progressively increasing the node count until the original graph size of 1,946 nodes is restored.

The models are trained for up to 7,500 epochs using the Adam optimizer with an initial learning rate of 10^{-3} . The rate is adaptively decreased using learning rate scheduler, and we employ early-stopping and checkpoint-saving mechanisms to prevent overfitting. A summary of the hyperparameters is provided in Table 1.

Model accuracy is evaluated using the MSE of unseen future snapshots, averaged over all nodes (N) and all future snapshots (S) being evaluated:

$$MSE = \frac{1}{S} \sum_{s=1}^S \left(\frac{1}{N} \sum_{n=1}^N (y_{n,s} - \hat{y}_{n,s})^2 \right), \quad (4)$$

where $\hat{y}_{n,s}$ and $y_{n,s}$ are the predicted and ground-truth values, respectively. This MSE formulation is also used as the loss function during model training. This study uses $S = 100$ until Section 5.2, and $S = 200$ from Section 5.3 onward to evaluate longer rollouts. While other

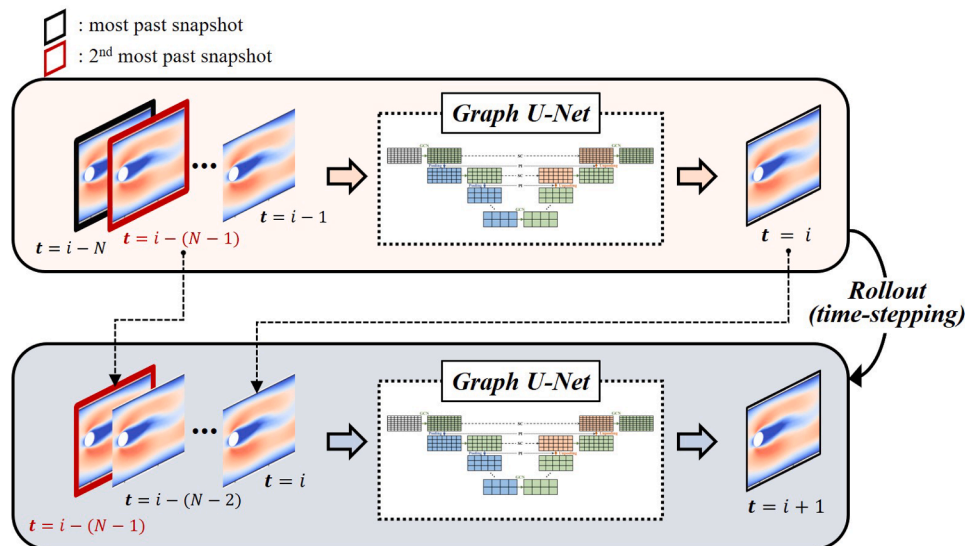


Fig. 3. Flowchart depicting how the rollout process is applied using the Graph U-Net: by using past N snapshots as input, Graph U-Net predicts the very next snapshot.

metrics like mean absolute error (MAE) (Stefenon et al., 2023) were also computed, their trends were highly consistent with MSE and are therefore omitted for conciseness.

5. Enhanced graph U-Nets for spatio-temporal flow prediction

5.1. Effects of convolutional operators

While the Graph U-Net paper (Gao & Ji, 2019) originally proposed using the GCN operator and GCN has demonstrated success in various graph-based tasks (De Avila et al., 2020; Hamilton et al., 2017; Ogoke et al., 2021; Schlichtkrull et al., 2018), it has known limitations in handling complex edge features (as noted in Section 3.1.2). This is a significant drawback for CFD problems, where edge features are crucial for capturing interactions between nodes. This section therefore compares different convolutional operators—GCN and GMM—to investigate whether the GMM's greater degrees of freedom in modeling edge features yield better performance in spatio-temporal flow prediction.

To investigate the impact of different GCN variants, we conduct experiments with three types: a standard GCN (without self-loop improvement), an improved GCN (iGCN) with self-loop enhancement ($A + 2I$) as proposed by Gao and Ji (2019), and an improved, weighted GCN (iwGCN) that additionally uses Euclidean distance for edge weights. We then compare these to three variants of the GMM operator with varying numbers of kernels: GMMs with 1, 2, and 3 kernels (referred to as GMM-1k, GMM-2k, and GMM-3k).

The results are presented in Table 2. As documented (Gao & Ji, 2019), the self-loop improvement enhances performance (iGCN vs. GCN), and incorporating Euclidean edge weights offers a further, though limited, benefit (iwGCN vs. iGCN). However, all GCN variants are significantly outperformed by the GMM operators, despite similar training times. Notably, GMM-1k achieves an MSE of 0.005×10^{-3} , a 95% improvement over the best GCN variant (iwGCN). This is attributed to the GMM operator's ability to model complex physics in the edge features using a learnable mixture of Gaussians. Interestingly, simply increasing the number of kernels does not guarantee better performance, as GMM-2k and GMM-3k yield higher MSEs than GMM-1k. This suggests a trade-off between model expressiveness and the risk of overfitting for this specific problem. The superior accuracy of GMM-1k over iwGCN is visually confirmed in Fig. 4. Based on these results, the single-kernel

Table 2

MSE ($\times 10^{-3}$) over 100 rollout predictions with varying convolutional operators and their training time (hours) based on NVIDIA 3080 GPU: three GCN-based variants with different options and three GMM-based variants with different number of kernels.

	Convolutional operator					
	GCN ¹	iGCN ²	iwGCN ³	GMM-1k ⁴	GMM-2k ⁵	GMM-3k ⁶
MSE	0.160	0.129	0.095	0.005	0.049	0.022
Time	0.84	0.82	0.84	0.81	0.83	0.84

¹ Vanilla GCN

² Improved GCN

³ Improved and weighted GCN

⁴ GMM with 1 kernel

⁵ GMM with 2 kernels

⁶ GMM with 3 kernels

GMM is used in subsequent experiments. The key takeaway in this section is that the choice of operator is critical for unsteady flow prediction. The traditional GCN operator may not be sufficient, and our results establish that architectures using GMM can more effectively represent spatio-temporal flow features.

In addition to prediction accuracy, computational efficiency during the inference phase is a critical factor for real-time applications. While the training process requires approximately 0.81 h for the GMM-1k model, the inference time for a single rollout step is approximately 1 ms. This rapid inference capability is orders of magnitude faster than the corresponding high-fidelity CFD simulations. This significant speedup, combined with the model's high accuracy, confirms the strong potential of the proposed Graph U-Net framework for real-time digital twin applications where immediate feedback is essential.

5.2. Effects of pooling ratio

In this section, we investigate how the pooling ratio—the proportion of nodes retained during the pooling operation—affects model performance. To determine the optimal degree of graph coarsening, we test Graph U-Net models with pooling ratios ranging from 0.1 to 0.9, as well as a variant without pooling/unpooling operations. The results of these experiments are shown in Table 3. It demonstrates that the choice of pooling ratio has a significant impact. The model without pooling performs poorly (MSE of 94.4×10^{-3}), which confirms that graph coarsening is beneficial for capturing multi-scale features in this architecture.

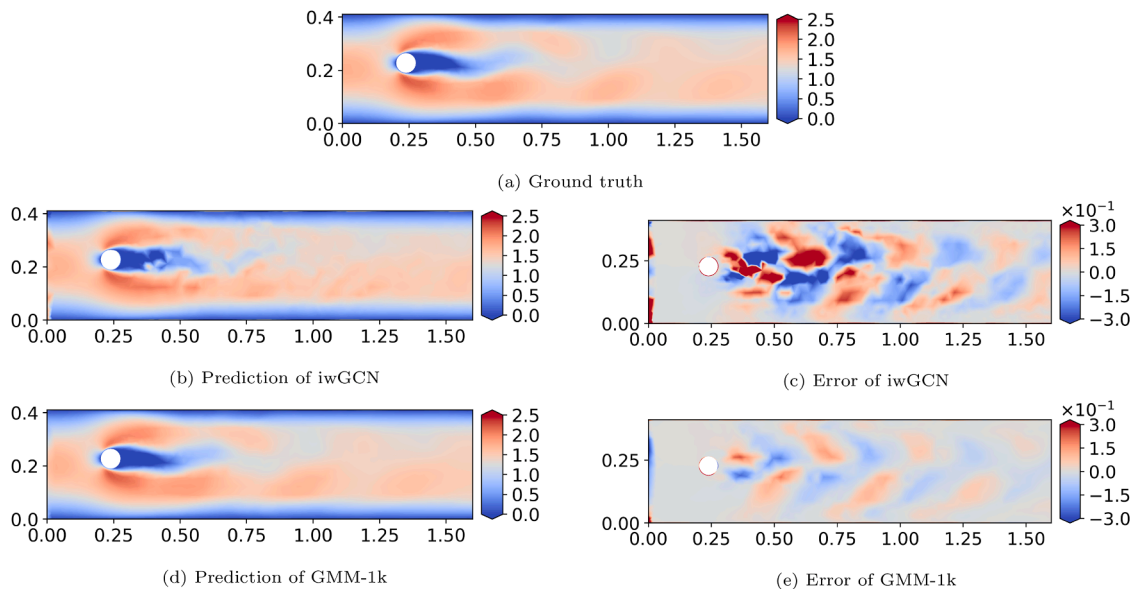


Fig. 4. Visualization of the x -velocity fields 50 snapshots after the trained snapshot range: iwGCN and GMM-1k operators are compared (all colorbars are expressed in units of m/s).

Table 3

MSE ($\times 10^{-3}$) over 100 rollout after the trained snapshot range and training time (hours) based on NVIDIA 3080 GPU as pooling ratio varies. Note that pooling ratio *None* indicates that the pooling/unpooling is not performed.

	Pooling ratio									
	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	None ¹
MSE	5.1	5.1	2.9	4.0	7.4	2.7	3.7	28.9	34.0	94.4
Time	0.77	0.76	0.77	0.81	0.81	0.85	0.89	0.94	1.02	0.57

¹ Without pooling/unpooling

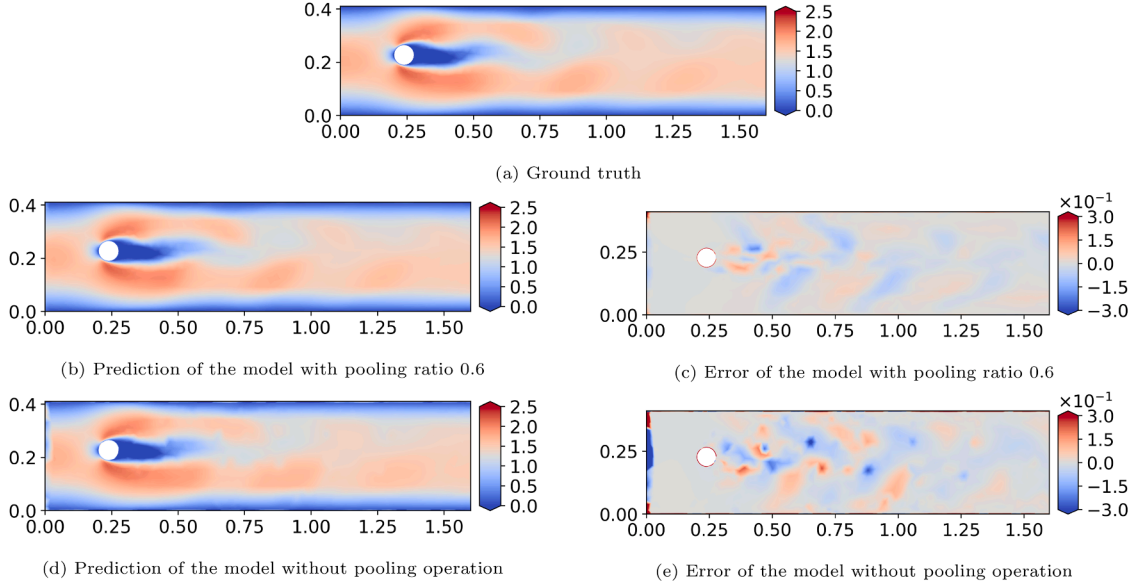


Fig. 5. Visualization of the x -velocity fields 100 snapshots after the trained snapshot range: predicted by the model with pooling ratio of 0.6 and model without pooling/unpooling operation (all colorbars are expressed in units of m/s).

A pooling ratio of 0.6 yields the best performance, suggesting this provides an optimal balance between graph coarsening and preserving spatial information for this problem. Increasing the pooling ratio beyond 0.7 dramatically degrades performance, likely because insufficient coarsening fails to capture critical physical information within the dense mesh.

Regarding computational cost, training time generally increases with the pooling ratio due to the denser graphs being processed during encoding/decoding. The model without pooling has the shortest training time, as it avoids the overhead of gPool/gUnpool operations. However, as shown in Fig. 5, this efficiency comes at a great cost to accuracy. While the model with a 0.6 pooling ratio produces successful results (Fig. 5(b)), the no-pooling model yields a noisy and inaccurate flow field (Fig. 5(d)). This trade-off underscores that the computational burden of pooling is justified by the significant gain in prediction accuracy.

To further illustrate the pooling operation, Fig. 6 visualizes the nodes retained after four pooling stages for ratios of 0.4 and 0.6. With a ratio of 0.4, the graph is coarsened aggressively, which may lead to the loss of important spatial information. An interesting point is that with the pooling ratio of 0.6, although the final pooled graph (Fig. 6(i)) is significantly sparser than the original mesh (Fig. 6(a)), the decoder still successfully restores the full-dimensional, accurate flow field, as shown in Fig. 5(b). This visualization of the pooled nodes in Fig. 6 reveals an intriguing pattern. Contrary to the expectation that nodes inside the vortex would be preserved, the resulting pooled nodes do not exhibit a clear, physically intuitive trend related to vortex shedding. They show an ambiguous pattern of alternating node clusters (Fig. 6(c)). Despite this lack of explicit physical meaning, it can be inferred that the pooling process captures an abstract representation of the vortex shedding phenomenon. To improve the interpretability of the gPool approach, future

work could explore extending the pooling method to directly incorporate flow quantities, such as velocity gradient or vorticity magnitude, instead of relying on a trainable projection vector p . Based on the findings above, we select a pooling ratio of 0.6 as the optimal value for the single-mesh experiments that follow.

5.3. Effects of noise injection

5.3.1. Motivation for noise injection

During inference, the auto-regressive nature of the model means its own, potentially noisy predictions are used as inputs for subsequent snapshots (Fig. 3). A discrepancy arises because the model is trained on clean, ground-truth data, yet it must perform inference on imperfect inputs. This can lead to a rapid accumulation of errors as the rollout progresses, causing the model's inputs to deviate from the training distribution (Sanchez-Gonzalez et al., 2020). To mitigate this issue and improve long-term prediction stability, we adopt a technique inspired by Sanchez-Gonzalez et al. (2020): injecting noise into the training data. This process is expected to make the model more robust by making the training distribution more similar to the distribution encountered during inference.

5.3.2. Application of noise injection approaches

Building on the work of Sanchez-Gonzalez et al. (2020), who injected noise into training inputs, we investigate two approaches:

1. **Input noise injection (I-noise):** In this approach, we corrupt only the input node features (x -velocity in this study) with Gaussian noise during training. This simulates the model receiving noisy data during an inference rollout.

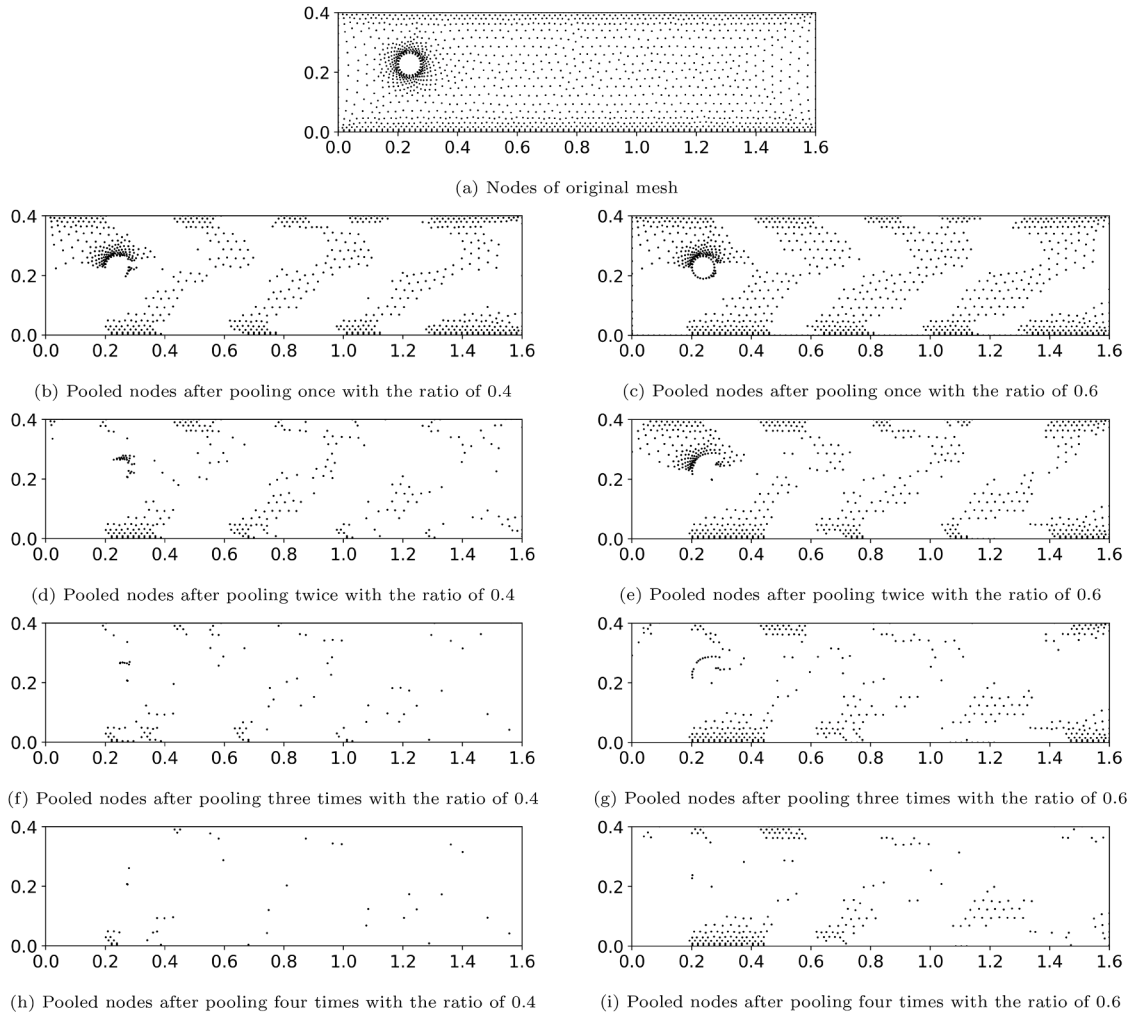


Fig. 6. Visualization of the pooled nodes at each level of the encoder with pooling ratios 0.4 (left column) and 0.6 (right column). The nodes of the original mesh are shown at the top.

2. Input-Output noise injection (I/O-noise): In addition to corrupting the input, we also inject noise into the output node features during training. This approach aims to prevent overfitting to the exact ground-truth snapshot, potentially encouraging the model to learn more generalized patterns.

To investigate their impact, we compare three models: one without noise injection, one with I-noise, and one with I/O-noise. The magnitude of the injected noise, σ from a Gaussian distribution $\mathcal{N}(0, \sigma^2)$, is a critical hyperparameter. We explore a range of noise sizes (σ): 0.001, 0.01, 0.02, 0.04, 0.08, 0.16, 0.32, and 0.64.

Table 4 presents the results over 200 rollout steps, an increased horizon to better assess long-term stability. Both I-noise and I/O-noise lead to remarkable improvements in stability compared to the without-noise model, provided an appropriate noise size is selected. This highlights that injecting noise during training significantly improves robustness to error accumulation. A noise size of $\sigma = 0.16$ yields the best performance for both approaches, with I-noise performing slightly better. At this level, the I-noise and I/O-noise models show an 86% and 85% improvement in MSE, respectively, over the model without noise. Fig. 7 visually compares an original flow field with one corrupted by the optimal noise size. It is noteworthy that training with this level of corruption (Fig. 7(b)) leads to superior performance. While sufficient perturbation enhances robustness, excessive noise (e.g., $\sigma \geq 0.32$) degrades performance, confirming the importance of tuning this hyperparameter.

Table 4

MSE ($\times 10^{-3}$) over 200 rollout after the trained snapshot range as noise injection approach varies. Note that the result without noise reports a single value since it does not depend on the noise size.

Noise type	Noise size							
	0.001	0.01	0.02	0.04	0.08	0.16	0.32	0.64
Without noise	6.26	–	–	–	–	–	–	–
I-noise	7.05	6.43	6.73	2.84	13.10	0.85	20.90	55.08
I/O-noise	9.10	4.65	5.10	2.24	16.47	0.92	11.75	48.02

ter. Based on these results, we select a noise size of 0.16 for subsequent experiments.

6. Transductive and inductive performance of enhanced graph U-Nets

This section evaluates the generalization capabilities of the enhanced Graph U-Nets, extending the analysis beyond the single training mesh discussed in Section 5. We assess both transductive performance—predicting on unseen node regions within a trained graph, and inductive performance—generalizing to entirely new, unseen graphs. Evaluating both settings is crucial for understanding the model's applicability to novel geometries and flow conditions encountered in real-world scenarios.

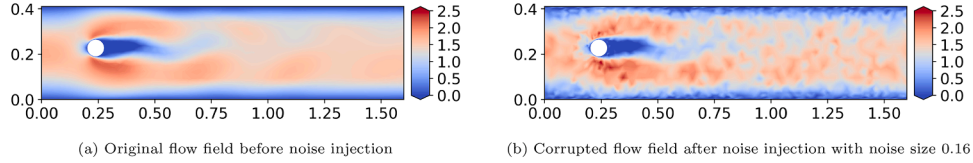


Fig. 7. Visual comparison of the flow fields before and after the noise injection: (a) without noise injection, (b) the best case with the noise size of 0.16. All colorbars are expressed in units of m/s .

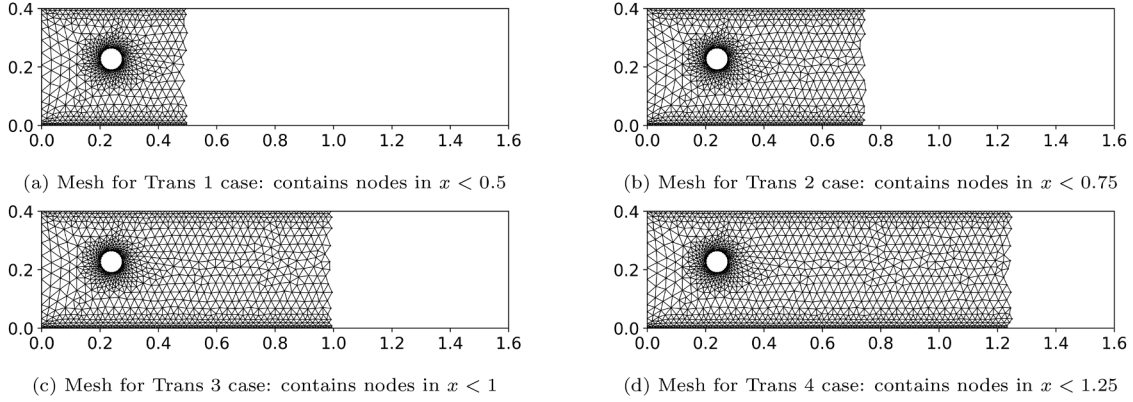


Fig. 8. Visualization of the meshes in four transductive cases: as the numbering of the cases increases, a larger portion of the x -domain is trained.

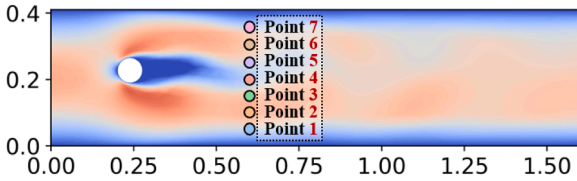


Fig. 9. The locations of seven probe points to be examined for a more intuitive understanding of how vortex shedding is predicted in terms of x -velocity.

6.1. Transductive-learning settings

This section investigates the model's transductive learning performance, i.e., its ability to predict flow quantities on unseen nodes within the training graph's domain. To evaluate this, we create four scenarios (Trans 1 to Trans 4) by progressively truncating the original mesh domain from Fig. 2 to include only nodes where $x < 0.5$ (Trans 1), $x < 0.75$ (Trans 2), $x < 1$ (Trans 3), and $x < 1.25$ (Trans 4), respectively, as shown in Fig. 8. The Graph U-Nets are trained on these truncated meshes and then tested on their ability to predict the flow field in the remaining, unseen downstream region. All models in this section use the I-noise approach with a noise size of 0.16. To visualize the results, the velocity profiles at seven probe points behind the cylinder, predicted up to 200 snapshots, will be plotted. Fig. 9 shows the locations of these points.

The time-series predictions for each case are shown in Fig. 10. The models trained on Trans 2, 3, and 4 demonstrate reasonable results, indicating that they learn the underlying vortex-shedding properties. However, the model trained on the Trans 1 case ($x < 0.5$) fails to capture the periodic behavior, as its training domain is too small to contain the full vortex region. In contrast, the Trans 2 case ($x < 0.75$), which includes the primary vortex region, shows a dramatic improvement. Fig. 11 further visualizes the performance of the Trans 2 model, showing that it accurately captures the temporal dynamics of vortex shedding and even successfully predicts the dissipation dynamics in the untrained region (wake region at $x > 0.75$). This highlights the strong transductive generalization of the proposed Graph U-Net framework, which can effectively predict quantities for unseen nodes by leveraging information from the trained graph structure.

6.2. Inductive-learning settings

To evaluate the inductive-learning performance of the improved Graph U-Nets, we consider three additional mesh scenarios, named Induct 1, Induct 2, and Induct 3, which are visualized in Appendix A. As summarized in Table 5, these scenarios feature different inlet velocities, cylinder geometries, and mesh properties compared to the baseline case (see Fig. A.1). Notably, the Induct 1 and Induct 2 cases denote fast-vortex-shedding phenomena similar to the baseline scenario, while Induct 3 is a slow-vortex-shedding case. This diversity in flow physics allows for a comprehensive evaluation of the model's generalization capabilities. Specifically, the variations in inlet velocity and cylinder diameter presented in Table 5 correspond to implicit changes in the Reynolds and Strouhal numbers, creating distinct flow regimes (e.g., fast vs. slow shedding). While this setup tests the model's ability to interpolate across these regimes, it is important to note the boundaries of this generalization; drastic changes in flow physics, such as a transition from laminar to fully turbulent flow, would likely necessitate retraining or transfer learning with data representative of those new regimes.

6.2.1. Zero-shot prediction

We first analyze the zero-shot prediction performance, where a model trained only on the baseline scenario is used to perform inference on the three unseen Induct cases. Here, zero-shot prediction means the rollout inference is performed on mesh scenarios that were never seen during training. The results in the lower part of Table 5 reveal a clear trend: both I-noise and I/O-noise models demonstrate reasonable accuracy for the fast-vortex shedding cases (Induct 1 and Induct 2) but exhibit significantly degraded performance for the slow-vortex shedding case (Induct 3). This discrepancy can be attributed to the model having learned only the fast-shedding dynamics present in the baseline training data.

The long-term zero-shot rollout performance is visualized in Fig. 12, which compares predicted x -velocity at seven probe points. As expected, the model performs best on the baseline case it was trained on (Fig. 12a). For the fast vortex shedding cases (Induct 1 and Induct 2), the model also demonstrates reasonable performance, although a slight drift in the shedding period becomes apparent as the rollout progresses (Fig. 12b and c). However, the model fails to accurately capture the phase and

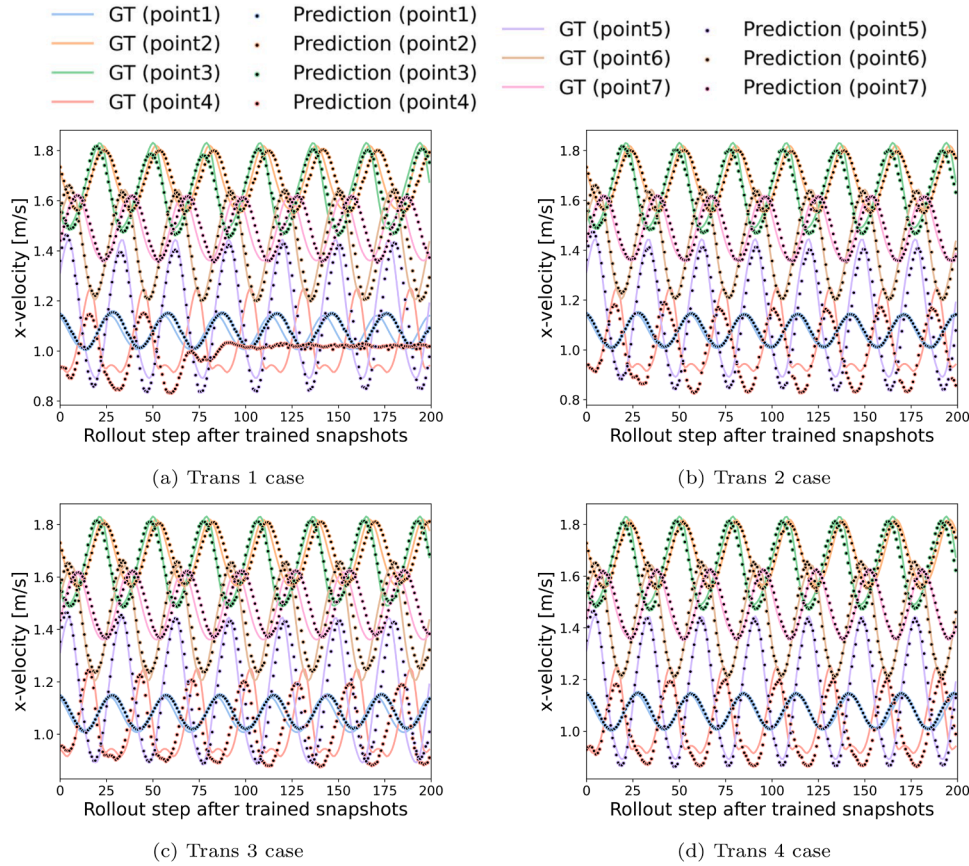


Fig. 10. Comparison of the predicted x -velocity at seven points (Fig. 9) in four transductive cases with I-noise approach. The velocity is predicted up to 200 snapshots (x -axis) after the trained snapshot range: (a) to (d) show the results from Trans 1 to Trans 4 cases.

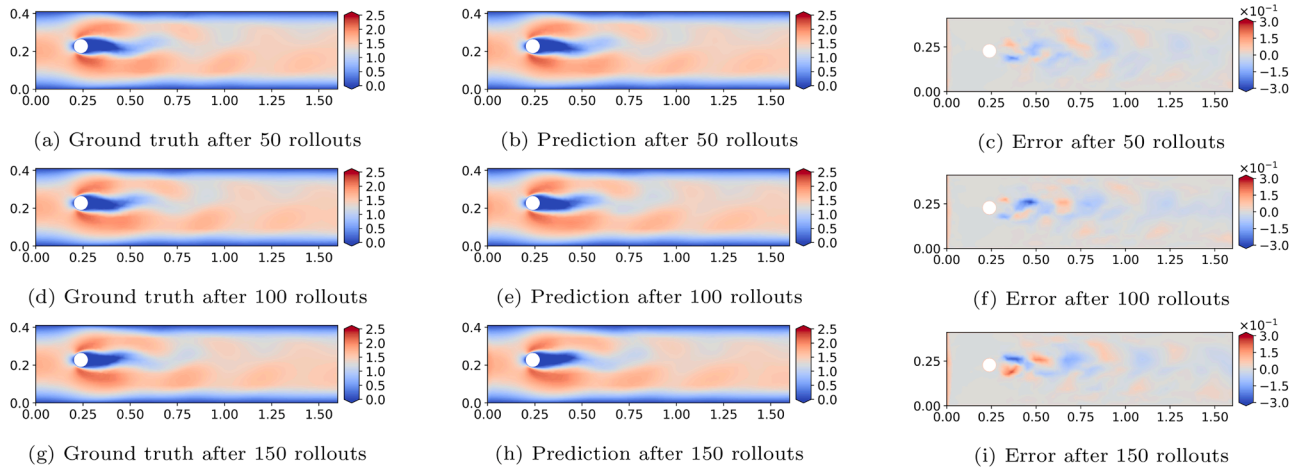


Fig. 11. Visualization of the x -velocity flow fields predicted by the model trained in Trans 2 case, where only the $x < 0.75$ region is trained. From top to bottom, each row shows the results of 50, 100, and 150 rollouts after the trained snapshot range. From left to right, each column shows the ground truth, prediction, and error.

frequency of the vortex shedding in the slower-shedding case (Induct 3), as evident in Fig. 12d. The predicted fluctuations clearly exhibit a shorter period than the ground truth, indicating that the model incorrectly applies the fast-shedding dynamics learned from the baseline scenario. These results demonstrate that while the Graph U-Net can generalize to unseen meshes with similar physical characteristics, its inductive performance is limited by the diversity of the training data, emphasizing the need to train on a wider range of flow characteristics to improve robustness.

6.2.2. Towards inductive learning

To overcome the limitations of the Graph U-Net models in generalizing to unseen meshes with different flow characteristics, we propose training a single model on a combination of meshes that exhibit both fast and slow vortex-shedding phenomena. By exposing the model to a wider range of flow dynamics, we aim to improve its inductive-learning performance. Specifically, we train a model on a combined dataset comprising two scenarios: the fast-shedding baseline case and the slow-shedding Induct 3 case.

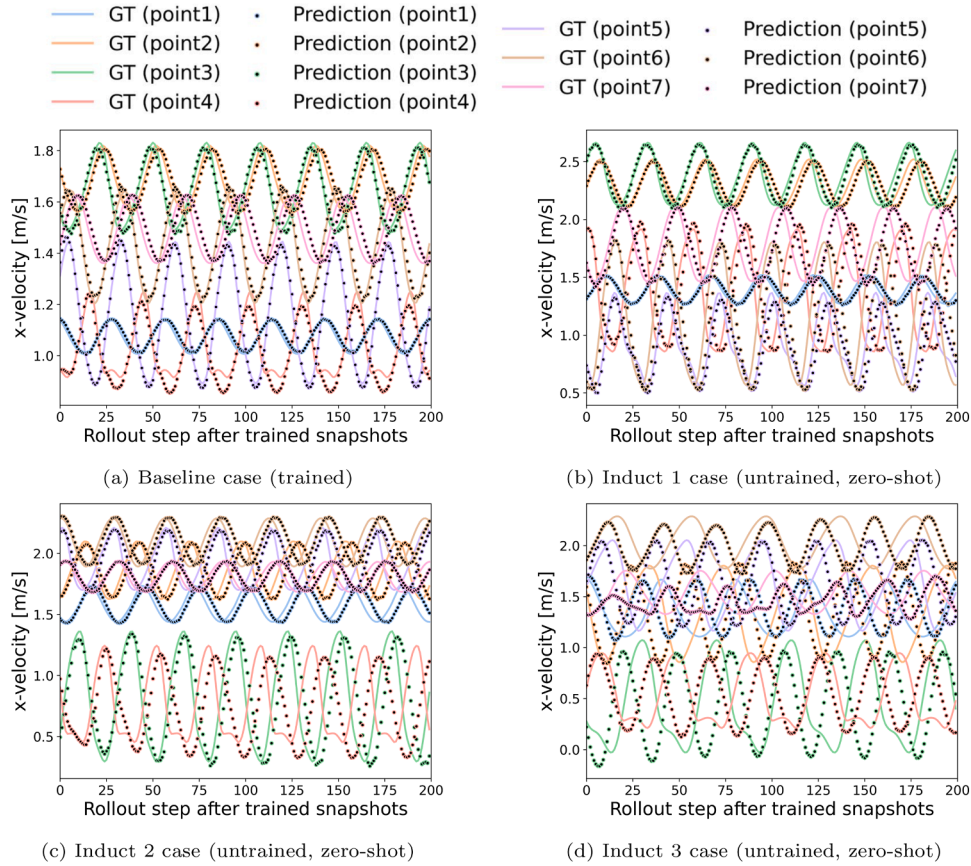


Fig. 12. Comparison of the predicted x -velocity at seven points (Fig. 9) when only the baseline mesh scenario is used for the training: (a) shows the prediction results of the trained baseline case while (b), (c), and (d) show the zero-shot results of the three inductive cases tested in this section. Induct 3 case shows significantly different periodic x -velocities compared with the other untrained cases, Induct 1 and Induct 2.

Table 5

Characteristics of the new mesh scenarios used to evaluate the inductive-learning performance (including the baseline scenario used in previous experiments). Also, MSE ($\times 10^{-3}$) over 200 rollout zero-shot predictions using the Graph U-Net trained only with the baseline mesh is presented: there is a clear trend that similar vortex-shedding cases (Induct 1 and Induct 2) have much higher accuracy than the Induct 3 case.

	Scenario type			
	Baseline	Induct 1	Induct 2	Induct 3
Shedding period	29	28	28	40
Inlet x -velocity* [m/s]	1.78	2.21	2.02	1.68
Cylinder diameter [m]	0.074	0.116	0.089	0.158
Number of nodes	1,946	1852	1925	1756
Number of edges	11,208	10,644	11,082	10,060
MSE (I-noise)	0.85	3.75	3.53	39.1
MSE (I/O-noise)	0.92	4.14	3.53	44.28

* Since the x -velocity profile of the inlet is assumed to be parabolic, only the midpoint of the profile, which is the maximum value of the inlet profile, is presented.

For this inductive-learning setting, while the overall architecture remains the same, several key hyperparameters are adjusted to handle the more complex training data: the number of input snapshots is increased to 40, and the number of GMM kernels is increased to 3. Consequently, the channel dimensions in the encoder's GMM operators change to [40, 20, 10, 5, 1]. We again employ I-noise and I/O-noise approaches with a noise size of 0.16 and include a without-noise case for comparison.

Table 6

MSE ($\times 10^{-3}$) over 200 rollout predictions in the inductive-learning setting is presented: different noise injection approaches (including without noise injection) and pooling ratios (including without pooling operation) are examined. Note that the Induct 1 and Induct 2 cases are zero-shot predictions.

Noise type	Pooling Ratio	Scenario type			
		Baseline [†]	Induct 1	Induct 2	Induct 3 [*]
I-noise	0.2	110.49	128.49	112.89	142.33
	0.4	95.40	78.38	101.10	71.52
	0.6	32.40	26.65	32.95	91.34
	0.8	2.74	7.41	6.33	41.05
	Without pooling	4.97	6.32	6.30	1.78
I/O-noise	0.2	103.68	117.97	114.30	142.94
	0.4	113.08	73.84	74.20	80.97
	0.6	4.68	17.33	12.76	94.16
	0.8	2.60	5.44	5.37	14.67
	Without pooling	5.32	6.31	6.30	1.86
Without noise	0.2	73.60	99.93	90.90	96.04
	0.4	70.83	85.62	89.04	146.64
	0.6	22.99	46.17	39.52	88.26
	0.8	66.48	82.24	106.76	149.36
	Without pooling	44.25	127.34	97.45	18.34

[†] Both Baseline and Induct 3 graphs are trained for this inductive-learning setting.

Further details on the choice of these hyperparameters can be found in Appendix B.

The results, summarized in Table 6, reveal a counter-intuitive finding: for inductive learning on multiple mesh scenarios, omitting the

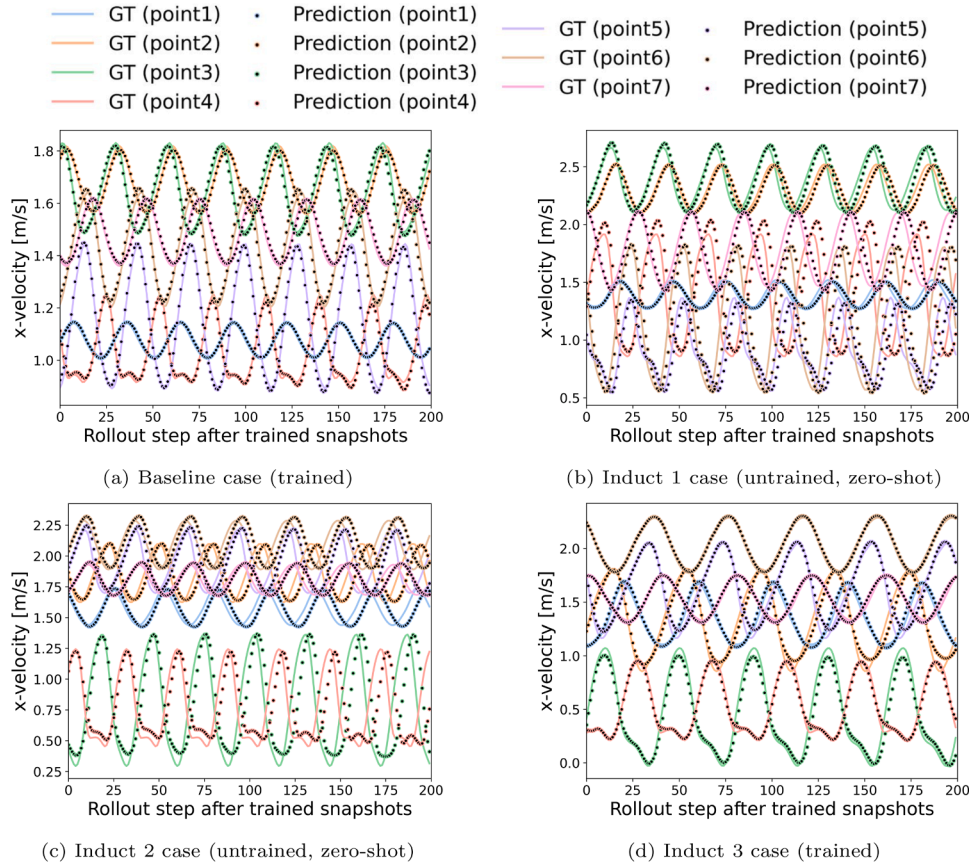


Fig. 13. Comparison of the predicted x -velocity at seven points (Fig. 9) when both the baseline and Induct 3 graphs are used for inductive learning. The results are from the model with I-noise and without pooling: (a) and (d) show the results of the trained case while (b) and (c) show the zero-shot results of the two inductive cases tested in this section.

pooling operations leads to dramatically better performance. This is in stark contrast to the single-mesh results where a pooling ratio of 0.6 was optimal. This outcome can be attributed to the inherent limitation of the gPool layer in a multi-graph context. The pooling mechanism learns a single projection vector to rank and select nodes, creating a unified strategy for graph coarsening. When meshes possess different structures and correspond to distinct physical regimes (e.g., fast vs. slow vortex shedding), a single learned pooling strategy is unlikely to be optimal for all of them. It may create a sub-optimal compromise that deteriorates the unique features of each graph, or overfit to one regime at the expense of another. By removing pooling, the model is forced to operate on the full, raw structure of each graph. This preserves the distinct topological information of each scenario, allowing the GMM convolutional layers greater flexibility to learn and adapt to the specific local dynamics of each case, which is critical for successful inductive generalization.

Fig. 13 shows the velocity profiles for the best-performing model (I-noise, without pooling). The model achieves satisfactory accuracy on the two trained cases (Fig. 13(a) and (d)) and also performs well on the unseen zero-shot cases (Fig. 13(b) and (c)). The stark improvement is visualized in Fig. 14. The model without inductive learning fails to capture the correct flow physics for the Induct 3 case (Fig. 14(b)), while the inductively trained model's prediction is very similar to the ground truth (Fig. 14(d)). Quantitatively, inductive training improves the MSE for the Induct 3 case by over 95%, reducing it from 39.1×10^{-3} with the zero-shot model to 1.78×10^{-3} . This remarkable 95% error reduction is achieved with only a linear increase in computational cost (i.e., doubling the training data), highlighting the efficient and powerful inductive learning capability of the proposed Graph U-Net framework.

It is crucial to emphasize that the remarkable finding here is not merely that adding slow-shedding data improves slow-shedding predic-

tion. The key result is that a single Graph U-Net model, when trained on past snapshots from different physical regimes, learns to identify the appropriate dynamics from the input history data and generate accurate, regime-specific future predictions, rather than simply averaging the learned behaviors. This demonstrates a robust capability to extrapolate in the time domain by conditioning on the temporal context of the input data.

6.2.3. Effects of normalization

As noted in Section 3.3.1, the models in this study primarily use layer normalization (LN). This section investigates the impact of this choice on inductive-learning performance by comparing three approaches: no normalization, LN, and Graph Normalization (GN). Recall that LN normalizes each node's features across the channel dimension (time dimension in this study), while GN normalizes each feature across the node dimension (space). The evaluation uses the best-performing inductive setting derived previously (training on the Baseline and Induct 3 cases without pooling) and considers both I-noise and I/O-noise approaches. The results are summarized in Table 7.

Interestingly, while GN outperforms LN for the baseline graph, LN exhibits superior accuracy for the dissimilar Induct 3 graph. With the I-noise approach, for example, the LN model achieves an MSE of 1.78×10^{-3} for the Induct 3 case, compared to 8.88×10^{-3} for the GN model. In this scenario, the GN results are even worse than using no normalization, which indicates that the ability to learn different physical behaviors in an inductive setting depends heavily on the normalization type. The results in Table 7 suggest that for the inductive setting where graphs with different physical characteristics are used for training, the LN approach is the most effective for achieving robust overall accuracy.

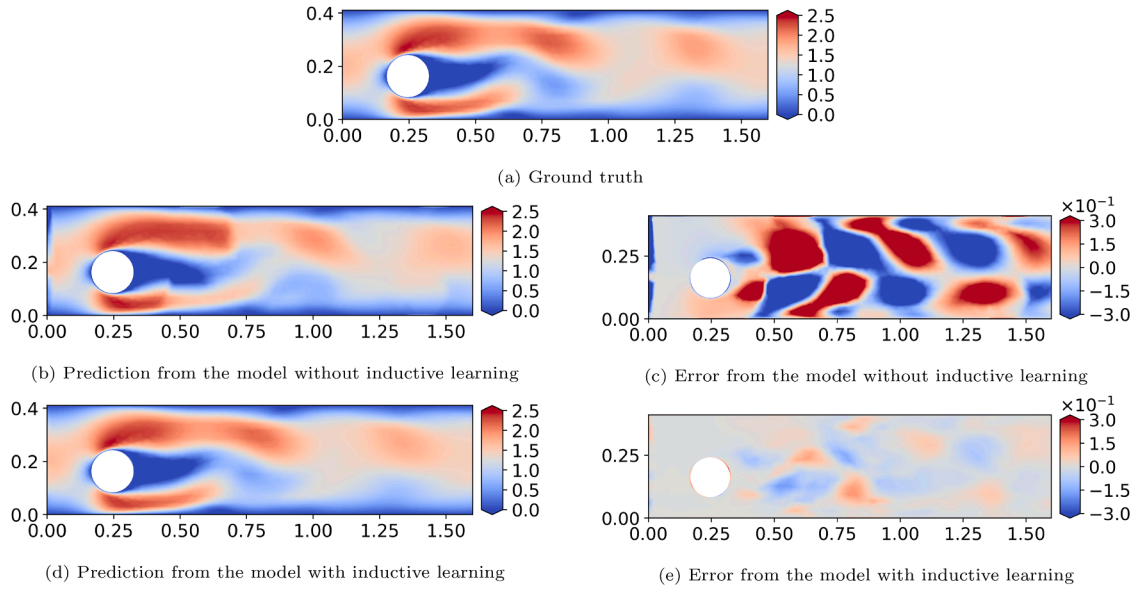


Fig. 14. For two models (without inductive setting and with inductive setting), x -velocity flow fields of 150 rollouts after the trained snapshot range in Induct 3 graph are compared. Both models are trained with I-noise (noise size of 0.16) and without pooling: (b) and (c) show the results without inductive learning and (d) and (e) show the results with inductive learning. All colorbars are expressed in units of m/s .

Table 7

MSE ($\times 10^{-3}$) over 200 rollout predictions is presented with varying normalization types in inductive setting: without normalization, LN (layer normalization), and GN (graph normalization). Results are shown for both I-noise and I/O-noise approaches, without pooling operation.

Description	Norm type	Scenario type			
		Baseline*	Induct 1	Induct 2	Induct 3*
I-noise & Without pooling	Without	4.24	9.47	7.73	5.95
	LN	4.97	6.32	6.30	1.78
	GN	1.82	6.55	6.02	8.88
I/O-noise & Without pooling	Without	4.77	9.73	8.07	5.62
	LN	5.32	6.31	6.30	1.86
	GN	2.20	6.76	6.35	13.24

* Both Baseline and Induct 3 graphs are trained for this inductive-learning setting.

This superior performance of LN can be attributed to its ability to focus on capturing temporal dynamics. By normalizing the node features (x -velocity in this study) across time for each node independently, LN ensures that the model can learn and adapt to the unique temporal patterns present at each node in the unsteady flow field. This is particularly important in our problem, where temporal evolution is the critical aspect of the prediction task. In contrast, GN normalizes node features across all nodes for each feature dimension, focusing on capturing the spatial dynamics between nodes. While this can be advantageous, it may be less crucial in our specific problem because the mesh of each scenario remains static over time. By normalizing across the node dimension, GN may unintentionally suppress the unique temporal patterns at individual nodes, as it considers the global statistics of the entire graph for each feature rather than focusing on individual node dynamics.

This comparative analysis highlights that the choice of normalization technique must be aligned with the problem's characteristics. In our study, where the goal is to predict temporal evolution on static meshes, LN proves to be superior. However, it should be noted that in scenarios where the meshes themselves vary with time, GN may outperform LN due to its ability to handle changing spatial dynamics within the graph. These findings underscore the importance of selecting an appropriate normalization approach that aligns with the specific requirements of the problem at hand.

6.2.4. Scalability and relation to operator learning

While our results demonstrate strong inductive generalization across different flow regimes, scaling graph-based models to industrial problems with millions of nodes remains a challenge. In this context, operator learning frameworks like graph neural operator (GNO) (Li et al., 2020) offer favorable scalability by treating data as continuous functions. However, these paradigms are not mutually exclusive; the proposed Graph U-Net can serve as a robust backbone within GNO frameworks. This integration enables the explicit capture of mesh connectivity—a key strength of our method—while leveraging the generalization performance of neural operators. Adopting such an operator-learning perspective can be particularly advantageous when the model requires robust generalization across a broad spectrum of flow conditions, such as wide ranges of Reynolds numbers.

7. Conclusions and future work

This study presented a comprehensive investigation of Graph U-Nets for unsteady, mesh-agnostic flow-field prediction, establishing a framework to adapt these models for complex regression tasks by introducing key architectural enhancements. We demonstrated that replacing the traditional GCN operator with a Gaussian mixture model (GMM) convolutional operator improves prediction accuracy by up to 95%, while incorporating noise injection enhances long-term rollout stability, reducing temporal prediction error by 86%. The model's generalization was confirmed in both transductive and inductive settings. For transductive learning, the model successfully predicted flow fields in unseen regions of a graph, even when trained on a mesh with the downstream domain truncated. For inductive learning across diverse physical scenarios, we found that optimal performance was achieved by adapting the architecture, specifically by omitting pooling operations and using layer normalization (LN), which allows the model to flexibly capture the unique dynamics of each graph.

Furthermore, as this framework matures for digital twin applications, it is important to consider its broad application scope and current limitations. The mesh-agnostic approach is relevant to a wide range of engineering domains, from environmental modeling and renewable energy system design to high-fidelity simulations in the aerospace industry. A key technical limitation for these demanding, large-scale applications is scalability. The computational cost in GNNs can become

a bottleneck for large-scale engineering problems with graphs containing millions of nodes and edges. A promising avenue to address this is to transition from graph-centric models to point-based approaches. Operator learning frameworks like DeepONet (Lu et al., 2021; Yang et al., 2025a) or Transformer-based architectures (Alkin et al., 2025; Luo et al., 2025), which treat the simulation data as an unstructured point cloud, could offer more favorable scaling properties. This introduces an important trade-off, as the gain in computational efficiency may come at the cost of predictive accuracy since these methods do not explicitly leverage the mesh connectivity information that GNNs are designed to capture. Investigating this balance between scalability and accuracy is a key direction for future research.

While this study provides a thorough optimization of the Graph U-Net framework, its validation was conducted on a 2D benchmark case. However, our inductive learning results demonstrated that the model could successfully generalize across distinct flow regimes—characterized by varying Reynolds and Strouhal numbers—by learning local nodal dynamics rather than memorizing global flow patterns. This capability suggests that the model is not overfitted to a specific geometry but captures the underlying local physics. Therefore, regarding alternative geometries such as airfoils or backward-facing steps, we theoretically expect the model to generalize effectively, provided that the GMM operator can leverage these learned local interactions to resolve boundary layer separations and wake formations inherent in those shapes.

Future work should therefore focus on extending this validation to more complex scenarios, including 3D turbulent flows and real-world experimental data, to fully assess the model's generalizability and robustness. Furthermore, our investigation focused on a deep-dive internal ablation of the Graph U-Net architecture itself. A rigorous benchmarking effort against other prominent models, such as MeshGraphNets or physics-informed neural networks (PINNs), represents another critical future direction to quantitatively assess its performance and computational trade-offs within the wider landscape of AI-driven fluid dynamics. Beyond broader validation, we will also extend the Graph U-Net architecture to be robust for much longer rollout predictions by applying various temporal numerical schemes (Yang et al., 2025b; Zhou & Farimani, 2025) or deep-learning architectures specialized for temporal predictions (Solera-Rico et al., 2024; Sanchis-Agudo et al., 2025). For example, applying the forward Euler method with Graph U-Nets that predict δu (the change in velocity u) instead of predicting u directly can be an option. In this context, these extensions can be implemented without modifying the core Graph U-Net architecture, by augmenting the current training procedure with multi-step rollout supervision (Wu et al., 2022). Finally, graph-based prediction will be coupled with an uncertainty-quantification approach (Morimoto et al., 2022; Yang & Yee, 2024), which can be attached on top of the existing predictor to estimate predictive confidence and enable Bayesian design optimization through adaptive sampling (Yang et al., 2023; Yang & Yee, 2022). Collectively, these research avenues aim to mature the proposed framework into a more robust and practical tool for accelerating design cycles in complex engineering systems.

CRediT authorship contribution statement

Sunwoong Yang: Supervision, Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization; **Ricardo Vinuesa:** Supervision, Writing – review & editing; **Namwoo Kang:** Supervision, Funding acquisition.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Ministry of Science and ICT of Korea grants (RS-2024-00355857, GTL24031-000, and N10250154), and the Ministry of Trade, Industry & Energy (RS-2025-25444634, RS-2025-02317327). RV was supported by ERC Grant 2021-CoG-101043998, DEEPCONTROL. The views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of European Union or European Research Council.

Appendix A. Visualization of the three meshes used for inductive learning (Section 6.2)

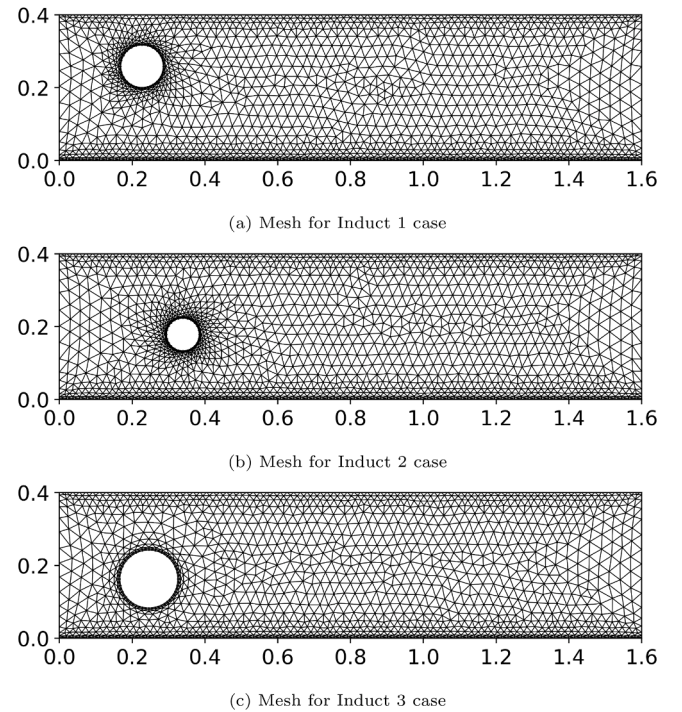


Fig. A.1. Three meshes used for the zero-shot prediction in Section 6.2.1 and inductive learning in Section 6.2.2.

Appendix B. Effects of the number of input snapshots and GMM kernels in inductive learning

The experiments in this section are based on the I-noise approach without pooling in inductive-learning setting (Section 6.2.2). In Table B.1, MSE of five different Graph U-Nets with varying number of input snapshots are compared (Srinivasan et al., 2019). The results demonstrate that for all graph types (Baseline, Induct 1–3), an input with 40 snapshots shows the best performance. For this reason, the number of input snapshots is set to 40 in Section 6.2.2. In Table B.2, MSE of different Graph U-Nets with different number of GMM kernels are compared. Although the GMM kernel with 1 was sufficient for the previous experiments before the inductive learning (as can be found in Table 2), it is discovered that for the inductive-learning setting, more GMM kernels are required to ensure sufficient model capacity. In fact, the GMM with three kernels shows a dramatic improvement in the Induct 3 scenario compared with the single-kernel case (from 68.71×10^{-3} to 1.78×10^{-3}).

Table B.1

MSE ($\times 10^{-3}$) over 200 rollout predictions in inductive learning with varying number of input snapshots: both Baseline and Induct 3 meshes are trained for this inductive-learning setting.

Number of input snapshots	Scenario type			
	Baseline	Induct 1	Induct 2	Induct 3
1	78.28	97.72	80.86	109.78
5	123.83	146.40	117.56	163.05
10	29.40	59.51	50.67	63.89
20	70.19	86.88	80.45	108.19
40	4.97	6.32	6.30	1.78

Table B.2

MSE ($\times 10^{-3}$) over 200 rollout predictions in inductive learning with two different numbers of GMM kernels: both Baseline and Induct 3 meshes are trained for this inductive-learning setting.

Number of GMM kernels	Scenario type			
	Baseline	Induct 1	Induct 2	Induct 3
1	3.15	7.00	6.47	68.71
3	4.97	6.32	6.30	1.78

References

- Alkin, B., Bleeker, M., Kurl, R., Kronlachner, T., Sonleitner, R., Dorfer, M., & Brandstetter, J. (2025). AB-UPT: Scaling neural cfd surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers. *arXiv:2502.09692*.
- Anand, V., Gupta, S., Koundal, D., & Singh, K. (2023). Fusion of u-net and CNN model for segmentation and classification of skin lesion from dermoscopy images. *Expert Systems with Applications*, 213, 119230.
- Ba, J. L., Kiro, J. R., & Hinton, G. E. (2016). Layer normalization. *arXiv:1607.06450*.
- Barnett, J., Farhat, C., & Maday, Y. (2023). Neural-network-augmented projection-based model order reduction for mitigating the kolmogorov barrier to reducibility. *Journal of Computational Physics*, 492, 112420.
- Barwey, S., Shankar, V., Viswanathan, V., & Maulik, R. (2023). Multiscale graph neural network autoencoders for interpretable scientific machine learning. *Journal of Computational Physics*, 495, 112537.
- Cai, T., Luo, S., Xu, K., He, D., Liu, T.-y., & Wang, L. (2021). Graphnorm: A principled approach to accelerating graph neural network training. In *International conference on machine learning* (pp. 1204–1215). PMLR.
- Chen, F., Sun, X., Wang, Y., Xu, Z., & Ma, W. (2024). Adaptive graph neural network for traffic flow prediction considering time variation. *Expert Systems with Applications*, 255, 124430.
- Clevert, D.-A., Unterthiner, T., & Hochreiter, S. (2015). Fast and accurate deep network learning by exponential linear units (elus). *arXiv:1511.07289*.
- Cremades, A., Hoyas, S., Deshpande, R., Quintero, P., Lellep, M., Lee, W. J., Monty, J. P., Hutchins, N., Linkmann, M., Marusic, I. et al. (2024). Identifying regions of importance in wall-bounded turbulence through explainable deep learning. *Nature Communications*, 15 (1), 3864.
- De Avila, B.-P. F., Economou, T., & Kolter, Z. (2020). Combining differentiable PDE solvers and graph neural networks for fluid flow prediction. In *International conference on machine learning* (pp. 2402–2411). PMLR.
- Deng, H., Zhu, X., Wang, Y., & Yang, X. (2024). Discovering message passing hierarchies for mesh-based physics simulation. *arXiv:2410.03779*.
- Deng, L., Bao, W., Wang, Y., Yang, Z., Zhao, D., Wang, F., Bi, C., & Guo, Y. (2022). Vortex-U-Net: An efficient and effective vortex detection approach based on U-Net structure. *Applied Soft Computing*, 115, 108229.
- Eivazi, H., Le Clainche, S., Hoyas, S., & Vinuesa, R. (2022a). Towards extraction of orthogonal and parsimonious non-linear modes from turbulent flows. *Expert Systems with Applications*, 202, 117038.
- Eivazi, H., Tahani, M., Schlatter, P., & Vinuesa, R. (2022b). Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations. *Physics of Fluids*, 34 (7).
- Feng, T., Wang, C., Chen, X., Fan, H., Zeng, K., & Li, Z. (2021). URNet: A U-Net based residual network for image dehazing. *Applied Soft Computing*, 102, 106884.
- Gao, H., & Ji, S. (2019). Graph U-Nets. In *International conference on machine learning* (pp. 2083–2092). PMLR.
- Gao, Z., Li, Z., Xu, L., & Yu, J. (2023). Dynamic adaptive spatio-temporal graph neural network for multi-node offshore wind speed forecasting. *Applied Soft Computing*, 141, 110294.
- Guastoni, L., Güemes, A., Ianaro, A., Discetti, S., Schlatter, P., Azizpour, H., & Vinuesa, R. (2021). Convolutional-network models to predict wall-bounded turbulence from wall quantities. *Journal of Fluid Mechanics*, 928, A27.
- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems*, 30.
- Hasegawa, K., Fukami, K., Murata, T., & Fukagata, K. (2020). Machine-learning-based reduced-order modeling for unsteady flows around bluff bodies of various shapes. *Theoretical and Computational Fluid Dynamics*, 34, 367–383.
- Hu, J.-W., & Zhang, W.-W. (2022). Mesh-ConV: Convolution operator with mesh resolution independence for flow field modeling. *Journal of Computational Physics*, 452, 110896.
- Hu, L., Xiang, Y., Zhang, J., Shi, Z., & Wang, W. (2022). Aerodynamic data predictions based on multi-task learning. *Applied Soft Computing*, 116, 108369.
- Jeon, J., Lee, J., Vinuesa, R., & Kim, S. J. (2024). Residual-based physics-informed transfer learning: A hybrid method for accelerating long-term CFD simulations via deep learning. *International Journal of Heat and Mass Transfer*, 220, 124900.
- Jurado, X., Reiminger, N., Benmoussa, M., Vazquez, J., & Wemmer, C. (2022). Deep learning methods evaluation to predict air quality based on computational fluid dynamics. *Expert Systems with Applications*, 203, 117294.
- Kang, Y.-E., Yang, S., & Yee, K. (2022). Physics-aware reduced-order modeling of transonic flow via β -variational autoencoder. *Physics of Fluids*, 34(7).
- Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. *arXiv:1609.02907*.
- LeCun, Y., Bottou, L., Orr, G. B., & Müller, K.-R. (2002). Efficient backprop. In *Neural networks: Tricks of the trade* (pp. 9–50). Springer.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2020). Neural operator: Graph kernel network for partial differential equations. *arXiv:2003.03485*.
- Lino, M., Fotiadis, S., Bharath, A. A., & Cantwell, C. D. (2022). Multi-scale rotation-equivariant graph neural networks for unsteady eulerian fluid dynamics. *Physics of Fluids*, 34 (8).
- Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G. E. (2021). Learning nonlinear operators via deepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3 (3), 218–229.
- Luo, H., Wu, H., Zhou, H., Xing, L., Di, Y., Wang, J., & Long, M. (2025). TranSolver + + : An accurate neural solver for PDEs on million-scale geometries. *arXiv:2502.02414*.
- Maulik, R., Lusch, B., & Balaprakash, P. (2021). Reduced-order modeling of advection-dominated systems with recurrent neural networks and convolutional autoencoders. *Physics of Fluids*, 33 (3).
- Monti, F., Boscaini, D., Masci, J., Rodola, E., Svoboda, J., & Bronstein, M. M. (2017). Geometric deep learning on graphs and manifolds using mixture model CNNs. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 5115–5124).
- Morimoto, M., Fukami, K., Maulik, R., Vinuesa, R., & Fukagata, K. (2022). Assessments of epistemic uncertainty using Gaussian stochastic weight averaging for fluid-flow regression. *Physica D: Nonlinear Phenomena*, 440, 133454.
- Multiphysics, C. (2015). Comsol multiphysics®, v. 5.4. Comsol AB, Stockholm, Sweden.
- NVIDIA Corporation (2025). Vortex shedding using mesh graph net. https://docs.nvidia.com/deeplearning/physicsemo/physicsemo-core/examples/cfd/vortex_shedding_mgn/readme.html. Accessed: 2025-07-05.
- Ogoke, F., Meidani, K., Hashemi, A., & Farimani, A. B. (2021). Graph convolutional networks applied to unstructured flow field data. *Machine Learning: Science and Technology*, 2 (4), 045020.
- Patel, V. A. (1978). Karman vortex street behind a circular cylinder by the series truncation method. *Journal of Computational Physics*, 28 (1), 14–42.
- Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., & Battaglia, P. W. (2020). Learning mesh-based simulation with graph networks. *arXiv:2010.03409*.
- Pichi, F., Moya, B., & Hesthaven, J. S. (2024). A graph convolutional autoencoder approach to model order reduction for parametrized PDEs. *Journal of Computational Physics*, 501, 112762.
- Qu, X., Liu, Z., An, W., Lyu, H., & Liu, X. (2022). Unsteady flow field prediction: Understanding the dynamics in an intuitive physics way. In *2022 26th international conference on pattern recognition (ICPR)* (pp. 1879–1885). IEEE.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III 18* (pp. 234–241). Springer.
- Roznowicz, D., Stabile, G., Demo, N., Fransos, D., & Rozza, G. (2024). Large-scale graph-machine-learning surrogate models for 3D-flowfield prediction in external aerodynamics. *Advanced Modeling and Simulation in Engineering Sciences*, 11 (1), 6.
- Sanchez-Gonzalez, A., Godwin, J., Pfaff, T., Ying, R., Leskovec, J., & Battaglia, P. (2020). Learning to simulate complex physics with graph networks. In *International conference on machine learning* (pp. 8459–8468). PMLR.
- Schlichtkrull, M., Kipf, T. N., Bloem, P., Van Den Berg, R., Titov, I., & Welling, M. (2018). Modeling relational data with graph convolutional networks. In *The semantic web: 15th international conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, proceedings 15* (pp. 593–607). Springer.
- Solera-Rico, A., Sanmiguel Vila, C., Gómez-López, M., Wang, Y., Almashjary, A., Dawson, S. T. M., & Vinuesa, R. (2024). β -variational autoencoders and transformers for reduced-order modelling of fluid flows. *Nature Communications*, 15 (1), 1361.
- Song, Q., Yao, W., Tian, H., Guo, Y., Muniyandi, R. C., & An, Y. (2024). Two-stage framework with improved u-net based on self-supervised contrastive learning for pavement crack segmentation. *Expert Systems with Applications*, 238, 122406.
- Srinivasan, P. A., Guastoni, L., Azizpour, H., Schlatter, P., & Vinuesa, R. (2019). Predictions of turbulent shear flows using deep neural networks. *Physical Review Fluids*, 4 (5), 054603.
- Stefanon, S. F., Seman, L. O., Mariani, V. C., & dos Santos, C. L. (2023). Aggregating prophet and seasonal trend decomposition for time series forecasting of italian electricity spot prices. *Energies*, 16 (3), 1371.
- Tang, M., Liu, Y., & Durlafsky, L. J. (2020). A deep-learning-based surrogate model for data assimilation in dynamic subsurface flow problems. *Journal of Computational Physics*, 413, 109456.
- Ulyanov, D., Vedaldi, A., & Lempitsky, V. (2016). Instance normalization: The missing ingredient for fast stylization. *arXiv:1607.08022*.

- Vinuesa, R. (2024). Perspectives on predicting and controlling turbulent flows through deep learning. *Physics of Fluids*, 36 (3).
- Sanchis-Agudo, M., Wang, Y., Arnau, R., Guastoni, L., Lim, J., Duraisamy, K., Vinuesa, R. (2025). Easy attention: A simple attention mechanism for temporal predictions with transformers. *APL Computational Physics*, 1 (1).
- Wang, L., Fournier, Y., Wald, J.-F., & Mesri, Y. (2024). Identification of vortex in unstructured mesh with graph neural networks. *Computers & Fluids*, 268, 106104.
- Wu, G., Al-qaness, M. A. A., Al-Alimi, D., Dahou, A., Abd Elaziz, M., & Ewees, A. A. (2024). Hyperspectral image classification using graph convolutional network: A comprehensive review. *Expert Systems with Applications*, 257, 125106.
- Wu, T., Wang, Q., Zhang, Y., Ying, R., Cao, K., Sosic, R., Jalali, R., Hamam, H., Maucec, M., & Leskovec, J. (2022). Learning large-scale subsurface simulations with a hybrid graph network simulator. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 4184–4194).
- Xiang, Z., Peng, W., Yao, W., Liu, X., & Zhang, X. (2024). Solving spatiotemporal partial differential equations with physics-informed graph neural network. *Applied Soft Computing*, 155 (p. 111437).
- Yang, S., Kim, H., Hong, Y., Yee, K., Maulik, R., & Kang, N. (2024). Data-driven physics-informed neural networks: A digital twin perspective. *Computer Methods in Applied Mechanics and Engineering*, 428, 117075.
- Yang, S., Lee, S., & Yee, K. (2023). Inverse design optimization framework via a two-step deep learning approach: Application to a wind turbine airfoil. *Engineering with Computers*, 39 (3), 2239–2255.
- Yang, S., Lee, Y., & Kang, N. (2025a). Data-efficient deep operator network for unsteady flow: A multi-fidelity approach with physics-guided subsampling. *Computer Methods in Applied Mechanics and Engineering*, 446, 118254.
- Yang, S., Vinuesa, R., & Kang, N. (2025b). Model-agnostic AI framework with explicit time integration for long-term fluid dynamics prediction. *Journal of Computational Design and Engineering*, 12 (10), 133–153.
- Yang, S., & Yee, K. (2022). Design rule extraction using multi-fidelity surrogate model for unmanned combat aerial vehicles. *Journal of Aircraft*, 59 (4), 977–991.
- Yang, S., & Yee, K. (2024). Towards reliable uncertainty quantification via deep ensemble in multi-output regression task. *Engineering Applications of Artificial Intelligence*, 132, 107871.
- Zhou, A., & Farimani, A. B. (2025). Predicting change, not states: An alternate framework for neural PDE surrogates. *Computer Methods in Applied Mechanics and Engineering*, 441, 117990.